

1. Omów mechanizm journalingu. Wymień nazwy systemów plików, w których jest stosowany.

- Inaczej księgowanie lub kronikowanie
- Rejestr operacji dyskowych
- Przechowuje informacje o wszystkich operacjach plikowych umożliwiając ich odtwarzalność
 - Po operacji tworzony jest odpowiedni wpis w dzienniku
- Dane nie są od razu zapisywane na dysk, tylko najpierw w kronice.
- Zmniejsza się prawdopodobieństwo utraty danych
 - Błąd w trakcie zapisywania - dokończony później (np. po podzyskaniu zasilania)
 - Błąd przed zapisem - oryginalny plik nietknięty
- Ext3/4, NTFS, Reiser4, ReiserFS, XFS, JFS, HFS+
- Metadata-only journaling - księgowanie wymaga zdwojenia operacji zapisu, więc księguje się tylko metadane plików

2. Omów proces uruchamiania komputera z BIOSem (od wciśnięcia przycisku zasilania do załadowania systemu operacyjnego).

- Z pamięci rzeczywistej procesor odczytuje adres pierwszej instrukcji BIOSu do wykonania
- Testy komponentów systemu (tzw etap POST - **Power On Self Test**). Piski w przypadku błędów
- BIOS skanuje pamięć w poszukiwaniu BIOSów innych urządzeń - łąduje je.
- Test pamięci i sprawdzenie zainstalowanych dysków
- BIOS odczytuje rekord MBR z dysku i zapisuje go w pamięci RAM w obszarze o adresie 0000:7C00H i wykonuje skok pod ten adres
 - MBR zawiera krótki program, którego zadaniem jest załadowanie systemu operacyjnego (IPL)
- uruchomiony program przegląda najpierw tablicę partycji (maksymalnie cztery partycje podstawowe), poszukując w niej partycji oznaczonej jako aktywna
- W przypadku komputerów z wieloma systemami operacyjnymi (multiboot) ładowany jest VBR - *Volume Boot Record*, który łąduje system operacyjny na partycji z załadowanym VBR (mechanizm chain load) (Np GRUB - stage 1 uruchamiany z MBR łąduje GRUB - stage 2 VBR z którego wybieramy Windowsy i łąduje się NTLDR i z niego wybieramy Windows XP)
- W Viście BOOTMGR odczytuje dane z Boot Configuration Data (dawniej **boot.ini**)
- Sektor łądzący z aktywnej partycji zostaje załadowany do tego samego obszaru pamięci w którym był MBR (bo są podobne)

- Na początku sektora ładującego umieszczony jest rozkaz skoku do sekwencji rozkazów inicjujących ładowanie systemu operacyjnego
- **Bootloadery: NTLDR (MS), LILO, GRUB (Linux)**

3. Wyjaśnij różnicę między liniowym a fizycznym adresem w pamięci operacyjnej?

- adres liniowy jest formą przejściową między adresem logicznym, a adresem fizycznym.
 - **translacja segmentacji** - adresy logiczne (złożone z selektora segmentu i przesunięcia) są przekształcane w adres liniowy
 - **translacja stronicowania** - adresy liniowe są przekształcane w adresy fizyczne. Opcjonalne, w zależności od konstrukcji systemu.
- Dzięki istnieniu adresu liniowego można uzyskać ciągłość pamięci nawet, jeżeli fizycznie jest ona porzucana.
 - Pozwala także na zaimplementowanie mechanizmu pamięci wirtualnej opartej na wymianie stron
- Adres liniowy może być równoznaczny adresowi fizycznemu (jeżeli wyłączymy tryb pracy wirtualnej)
- Adres liniowy jest zamieniany przez mechanizm stronicowania na adres fizyczny za pomocą tablic stron.

4. Wymień metody ukrywania danych (plików) w systemach plików i opisz pokrótce ideę każdej z nich.

- **In-band**
 - Wykorzystanie miejsca na dysku, które w każdej chwili może zostać nadpisane przez SO
- **Out-of-band**
 - Zapisujemy w miejscach na dysku, które nie mogą być zmienione (wykorzystując np. to że przez zaokrąglanie niektóre części dysku są nieużywane).
- Alternatywny strumień danych
 - każdy plik na partycji NTFS, obojętnie czy jest zwykłym plikiem czy katalogiem, posiada główny (nienazwany, unnamed) strumień, w którym zapisane są informacje, które przechowuje. To to jest to, co widzimy jak otwieramy plik. Plik może posiadać jednak więcej strumieni i można w nich przechowywać dane tak jak w normalnych plikach. Nie różnią się bowiem one niczym od nienazanego strumienia oprócz tego, że mają nazwy

5. Podaj cele stosowania struktury GDT(Global Descriptor Table).

- Procesy mają dostęp tylko do przydzielonych segmentów danych
- W tablicy GDT znajdują się dekryptory segmentów z polem , które określa poziom uprzywilejowania jaki trzeba mieć by do danego segmentu się dostać
- Proces (mający swój kod w jakimś segmencie, który ma odpowiedni poziom uprzywilejowania) przez selektor wskazuje na dekryptor.
 - Sprawdzane są poziomy uprzywilejowania selektora i dekryptora ==

BEZPIECZEŃSTWO

- **Możliwe wykorzystanie mechanizmu pamięci wirtualnej WIRTUALIZACJA PAMIĘCI**
- **Mapowanie jednego segmentu na różne zadania** (np. 2 procesy mogą mieć dostęp do tego samego segmentu danych)
- w slajdach na pewno było też o tym, że dwa procesy mogą używać ten sam adres liniowy, a poprzez translację(GDT, Page Directory, Page Table) - różne obszary pamięci.

6. Wyjaśnij co zawiera i jaką rolę pełni i-węzeł w systemie plików ext2/ext3?

- Pełni rolę struktury opisującej plik (katalog też jest plikiem) w systemie
- Wpis katalogowy danego pliku zawiera tylko jego nazwę i numer i-węzła (dowiązanie).
- Zwykle i-węzły przechowywane są w dużej tablicy na początku partycji dyskowej.
- Gdy plik jest otwierany to jego i-węzeł umieszczony jest w tablicy i-węzłów w pamięci operacyjnej
- System plików odwołuje się do i-węzłów poprzez indeks w tablicy i-węzłów
- Zawiera:
 - opis położenia pliku na dysku i rozmiar pliku;
 - identyfikator właściciela pliku;
 - typ pliku (pliki zwykłe, katalogi, . . .);
 - prawa dostępu do pliku (właściciela pliku, grupowego właściciela pliku, pozostałych użytkowników);
 - czasy dostępu do pliku (czas ostatniej modyfikacji, czas ostatniego dostępu, czas ostatniej modyfikacji i-węzła pliku);
 - liczba dowiązań do pliku;
 - liczba procesów, które otworzyły ten plik;
 - inne;
 - można zauważyć, że i-węzeł nie zawiera nazwy pliku !;

7. Opisz rolę bloku PEB (blok wykonawczy procesu).

zmieniasz pytanie ;P Process Environment Block (PEB) -- [stad](#), ale nic tam nie ma o tym milcz pedale kurwa

- PEB - Process Environment Block jest częścią Thread Information Block (TIB) zawierające informacje o wątku.
- Structure **PEB** (*Process Enviroment Block*) contains all *User-Mode* parameters associated by system with current process.

- PEB - Blok środowiska procesu, dostępny w trybie użytkownika
 - Zawiera wszystkie parametry z trybu użytkownika skojarzone przez system z procesem
 - Ustawiany podczas tworzenia obiektu procesu
- PEB also contains a **module list** of dlls, including the names and the addresses where they are loaded. Every PE file requires kernel32.dll, and it is loaded at the same address for all processes

za <http://www.milw0rm.com/papers/66> w sumie to bardzo ciekawe, PEB jest wykorzystywany do heap overflow :-)

The PEB (Process Environment Block) is a process specific area of userland memory that contains details about each running process. Since PEB information is user modifiable, it runs in the context of the process address space. Information contained in the PEB includes the image base address, heap information, loaded modules and defined environment variables amongst other things.

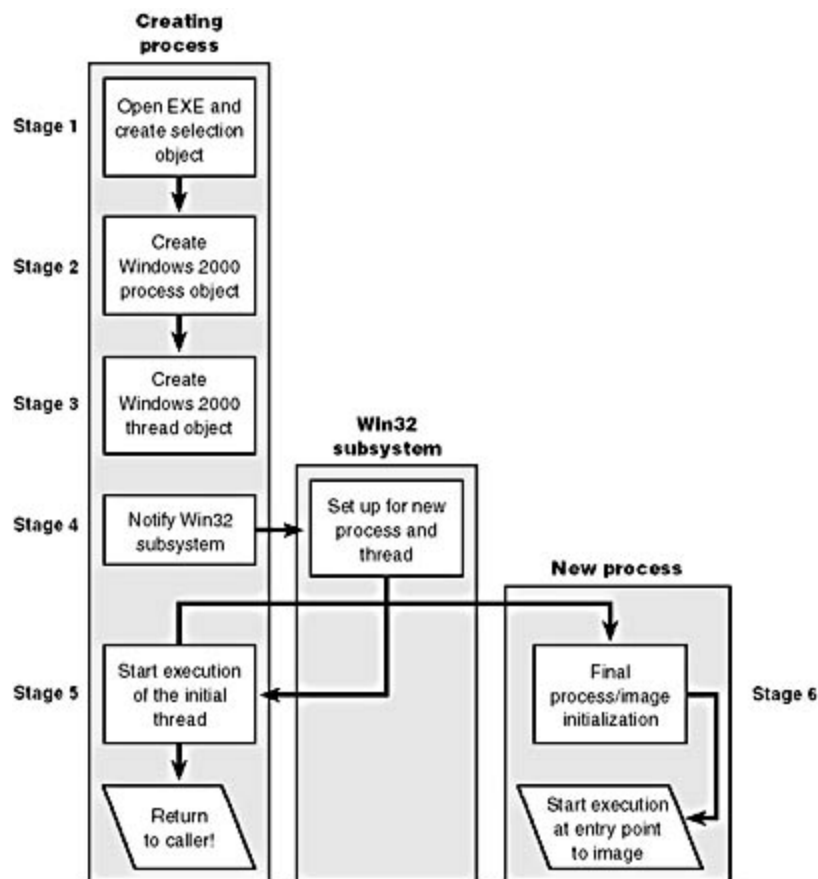
8. Opisz przepływ sterowania podczas tworzenia procesu w systemie Windows XP

<http://www.microsoft.com/mspress/books/sampchap/4354a.aspx>

The following list summarizes the main stages of creating a process with the Win32 *CreateProcess* function. The operations performed in each stage are described in detail in the subsequent sections.

1. Open the image file (.exe) to be executed inside the process.
 1. If the executable file specified is a Win32 .exe, it is used directly. If it's not a Win32 .exe (for example, if it's an MS-DOS, a Win16, a POSIX, or an OS/2 application), *CreateProcess* goes through a series of steps to find a Win32 support image to run it
2. Create the Windows 2000 executive process object.
 - A. Setting up the EPROCESS block
 - B. Creating the initial process address space
 - C. Creating the kernel process block
 - D. Concluding the setup of the process address space
 - E. Setting up the PEB
 - F. Completing the setup of the executive process object
3. Create the initial thread (stack, context, and Windows 2000 executive thread object).
 1. Before the thread can be created, it needs a stack and a context in which to run, so these are set up now.
4. Notify the Win32 subsystem of the new process so that it can set up for the new process and thread.
5. Start execution of the initial thread (unless the CREATE_SUSPENDED flag was specified).

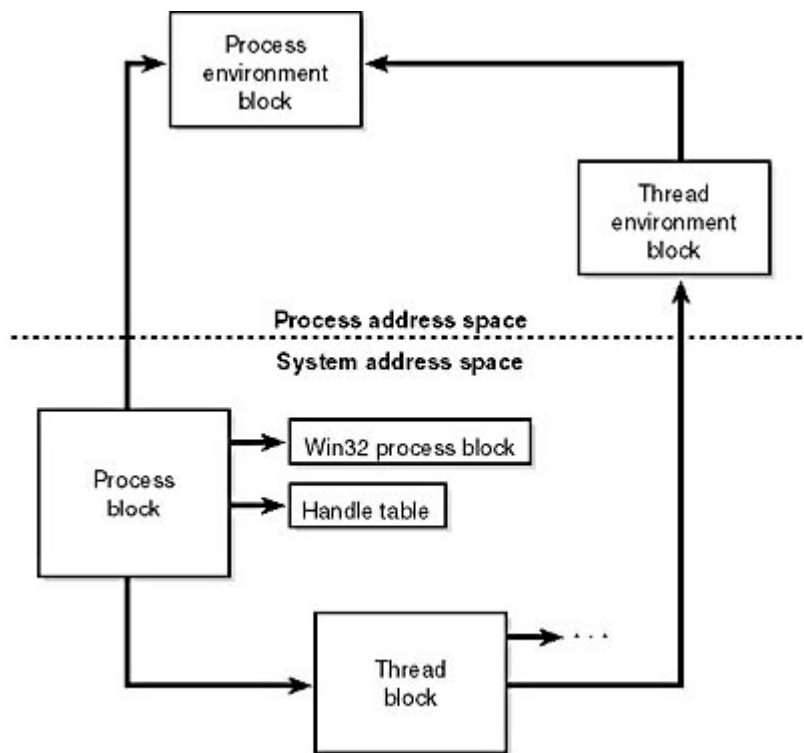
6. In the context of the new process and thread, complete the initialization of the address space (such as load required DLLs) and begin execution of the program.



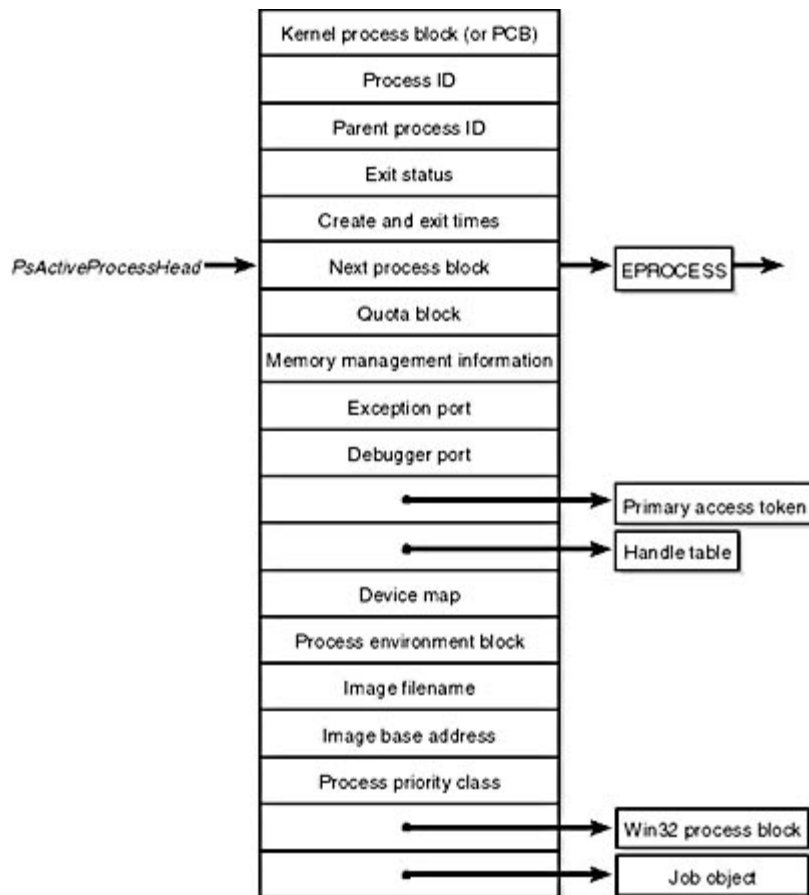
^raczej o to chodzi Tomkowi w tym pytaniu

- Otwarcie pliku .exe i utworzenie obiektów sekcji
- Utworzenie obiektów procesu
 - Ustawienie struktury EPROCESS (blok wykonawczy procesu, opisuje proces, parametry:[czas, id, id rodzica prev/next proces, wskaźnik na watek główny, PEB])
 - Utworzenie inicjalnej przestrzeni adresowej
 - Utworzenie bloku kernela procesu
 - Ukończenie tworzenia przestrzeni adresowej
 - Ustawienie PEB
 - Ukończenie konfiguracji EPROCESS
- Utworzenie obiektu wątku
- Powiadomienie podsystemu o nowym procesie
- Utworzenie wątku głównego
- Utworzenie kontekstu i uruchomienie wątku

Off-topic:



The EPROCESS block and its related data structures exist in system space, with the exception of the process environment block (PEB), which exists in the process address space (because it contains information that is modified by user-mode code)



9. Jaką rolę w systemie plików NTFS pełni nadrzędna tablica plików (MFT Master File Table)?

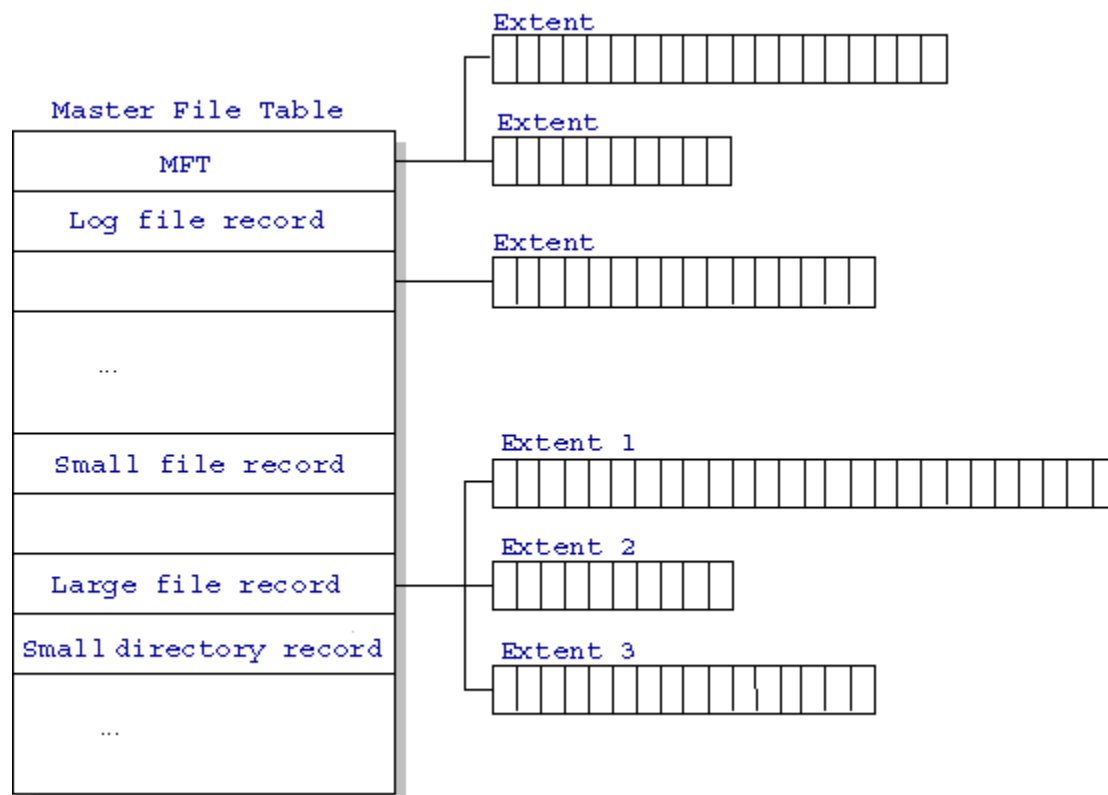
- Szybkość odczytu
- Odtwarzanie w razie awarii
- Journaling
- Umożliwiają hard links
- Zawiera atrybuty plików
- Ułatwia defragmentacje
- Bardzo podobne to tablicy i-węzłów
- Jeden z obaszarów partycji NTFS, ukryty plik
- Definiuje rozmieszczenie wszystkich plików na dysku
- Umieszczany na początku dysku
- Są w nim zawarte wpisy do każdego pliku na danej partycji (informacje o nim, parametry)

The Master File Table (MFT) contains [metadata](#) about every file, directory, and metafile on an NTFS volume. **It includes filenames, locations, size, and permissions.** Its structure **supports algorithms which minimize [disk fragmentation](#).** A directory entry consists of a filename and a "file ID" which is the record number representing the file in the Master File Table. The file ID also contains a reuse count to detect stale references. While this strongly resembles the W_FID of [Files-11](#), other NTFS structures radically differ.

Each file on an NTFS volume is represented by a record in a special file called the master file table (MFT). NTFS reserves the first 16 records of the table for special information. The first record of this table describes the master file table itself, followed by a MFT **mirror record**.

If the first MFT record is corrupted, NTFS reads the second record to find the MFT mirror file, whose first record is identical to the first record of the MFT. The locations of the data segments for both the MFT and MFT mirror file are recorded in the boot sector. A duplicate of the boot sector is located at the logical center of the disk.

The third record of the MFT is the log file, used for file recovery. The seventeenth and following records of the master file table are for each file and directory (also viewed as a file by NTFS) on the volume.



The master file table **allocates a certain amount of space for each file record.** The **attributes of a file are written to the allocated space in the MFT.** Small files and **directories** (typically 1500 bytes or smaller), such as the file illustrated in next figure, **can entirely be contained within the master file table record.**

Figure 5-2 MFT Record for a Small File or Directory:

Standard information	File or directory name	Security descriptor	Data or index	
----------------------	------------------------	---------------------	---------------	--

This design makes file access very fast. Consider, for example, the FAT file system, which uses a file allocation table to list the names and addresses of each file. FAT directory entries contain an index into the file allocation table. When you want to view a file, FAT first reads the file allocation table and assures that it exists. Then FAT retrieves the file by searching the chain of allocation units assigned to the file. With NTFS, as soon as you look up the file, it's there for you to use.

Directory records are housed within the master file table just like file records. Instead of data, directories contain index information. Small directory records reside entirely within the MFT structure. Large directories are organized into B-trees, having records with pointers to external clusters containing directory entries that could not be contained within the MFT structure.

10. Budowa i rola rekordu plikowego w NTFS.

- Każdy plik opisany jest w tablicy MFT w postaci **rekordu plikowego**
 - Rekord składa się z nagłówka oraz sekwencji występujących po sobie atrybutów w postaci pary
 - Każdy atrybut składa się z nagłówka (zawiera m.in. Typ i długość) i właściwych danych
 - Atrybuty dzieli się na
 - Rezydentne - ich rozmiar nie przekracza wielkości rekordu plikowego
 - Nierezydentne - nie mieszczą się w jednym rekordzie, są przechowywane poza tablicą MFT, w rekordzie umieszczane są wskaźniki doń
 - Nagłówek zawiera kilkanaście pól o różnym przeznaczeniu (m.in.. Faktyczny rozmiar rekordu i zaalokowany rozmiar na rekord), rozpoczyna się tekstem "FILE" w ASCII
 - Rekord zawiera:
 - Informacje standardowe (czas, liczba łączy, prawa dostępu)
 - Nazwa pliku
 - Informacje o woluminie
 - Dane pliku
 - Indeks główny (do implementacji katalogów)
 - Mapa bitowa

Table 3.6 NTFS File Attribute Types

Attribute Type Description	
Standard Information	Includes information such as time stamp and link count.
Attribute List	Lists the location of all the attribute records that do not fit in the MFT record.
File Name	A repeatable attribute for both long and short file names. The long name of the file can be up to 255 Unicode characters.

	The short name is the MS-DOS-readable, 8.3, case-insensitive name for the file. Additional names, or hard links, require additional file name attributes.
Security Descriptor	Shows information about who owns the file and who can access the file.
Data	Contains file data. NTFS allows multiple data attributes per file. Each file typically has one unnamed data attribute. A file can also have named data attributes, each using a particular syntax.
Object ID	A volume-unique file identifier. Used by the link tracking service. Not all files have object identifiers.
Logged Tool Stream	Similar to a data stream, but operations on a logged tool stream are logged to the NTFS log file just like NTFS metadata.
Reparse Point	Used for directory junction points and volume mount points. They are also used by file system filter drivers to mark areas of the file system.
Index Root	Used to implement folders and other indexes.
Index Allocation Bitmap	Used to implement folders and other indexes.
Volume Information	Used only in the \$Volume system file. Contains the volume version.
Volume Name	Used only in the \$Volume system file. Contains the volume name.

11. W jaki sposób tablica alokacji plików (FAT) opisuje położenie plików na dysku?

- partycja podzielona jest na klastry
 - Każdy klaster składa się z sektorów
 - Klastry są numerowane
 - Klaster w całości przydzielony jest jednemu plikowi
- SO na podstawie numeru klastra oblicza numer logiczny sektora (numer od początku partycji) - dzięki czemu może odczytać bądź zapisać sektor
- Tablica FAT to tablica opisująca każdy klaster na dysku (kolejny rekord tablicy to kolejny klaster)
 - Rekord ma wskaźnik do następnego rekordu opisującego dany plik lub zancznik oznaczający koniec plik.
 - Dzięki temu po liście można odczytać cały plik
- Katalogi traktowane są jak pliki i też mają swoje rekordy i klastry
 - W klastrze znajduje się specjalna struktura do opisu jakie pliki znajdują się w danym katalogu (identyfikowane przez numer wiersza w tablicy FAT)
 - Zawierają 2 dodatkowe wpisy: '.' i '..'

12. Wyjaśnij rolę superbloku w systemie EXT2/ext3

- Zawiera informacje o systemie plików jako całości
- Ma go każdy system plików (tzn. EXT2 i EXT3, bo znajduje się w grupie bloków charakterystycznej dla tych systemów plików. Konrad)
- Zawiera m. in. (by Garboś):
 - rozmiar bloku.
 - liczbie wszystkich oraz wolnych bloków i i-wezłów
 - stanie systemu plików (czy został poprawnie zamknięty)

- licznik zamontowania i data ostatniego sprawdzenia, pozwalają wymusić sprawdzenie
- integralności danych co jakiś czas.
- wersja systemu plików – zbiór 3 różnych kategorii rozszerzeń w zależności od stopnia wprowadzonych przez nie niezgodności. Są to:
 - **COMPAT** – zmiany należące do tej kategorii są zupełnie przejrzyste dla starszych wersji.
 - **RO COMPAT** – zmiany te nie powodują błędów przy odczycie, ale zapis w takim systemie przez wersje nie obsługujące rozszerzenia spowodowałoby uszkodzenie danych. Stara wersja może używać takiej partycji w trybie tylko do odczytu.
 - **INCOMPAT** – wprowadzone zmiany są na tyle poważne, że próba użycia systemu przez starsze wersje może zakończyć się błędem.

*Each block group may contain a copy of the superblock and block group descriptor table, and all block groups contain a block bitmap, an inode bitmap, an inode table and followed by the actual data blocks. The **superblock** contains important information that is crucial to the booting of the **operating system**, thus backup copies are made in multiple block groups in the file system. However, typically only the first copy of it, which is found at the first block of the file system, is used in the booting.*

13. Omówić strukturę partycji rozszerzonej na dysku

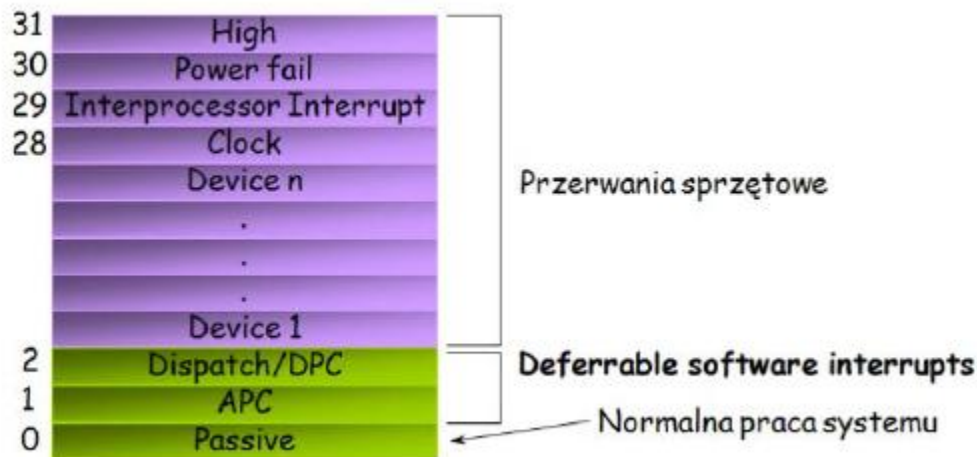
- Pojedyncza tablica partycji może zawierać max 4 dyski logiczne (partycje)
- Aby zwiększyć tę ilość można zamiast zwykłej partycji zrobić partycję rozszerzoną
 - W jej skład wchodzi kolejne dyski logiczne
 - na początku partycji rozszerzonej znajduje się sektor o specyficznej strukturze (**tzw. EBR**), w którym zawarta jest tablica partycji, położona w tym samym miejscu (offset) co w przypadku MBR; w tablicy tej wykorzystywane są tylko dwie pierwsze pozycje
 - Pierwszy element wskazuje adres początkowy dysku logicznego
 - Drugi element to wskaźnik na kolejny element listy dysków logicznych (jeśli 0 - koniec listy) - offset, odległość od początku partycji rozszerzonej
 - Dysk logiczny partycji rozszerzonej jest taki sam jak partycji podstawowej
 - Ma sektor ładujący partycji (informacje o partycjach, systemie plików)

14. Opisz mechanizm obsługi przerwania na poziomie jądra w systemie Windows XP. Wyjaśnij rolę zmiennej IRQL (**Interrupt request level**).

- Wszystkie przerwania obsługiwane są przez systemowy dispatcher, który ma swoją IDT (interrupt dispatch table). IDT dispatchera jest systemowym odpowiednikiem IDT

(interrupt descriptor table) zwykłego, procesorowego (to taka abstrakcja nad sprzętem).

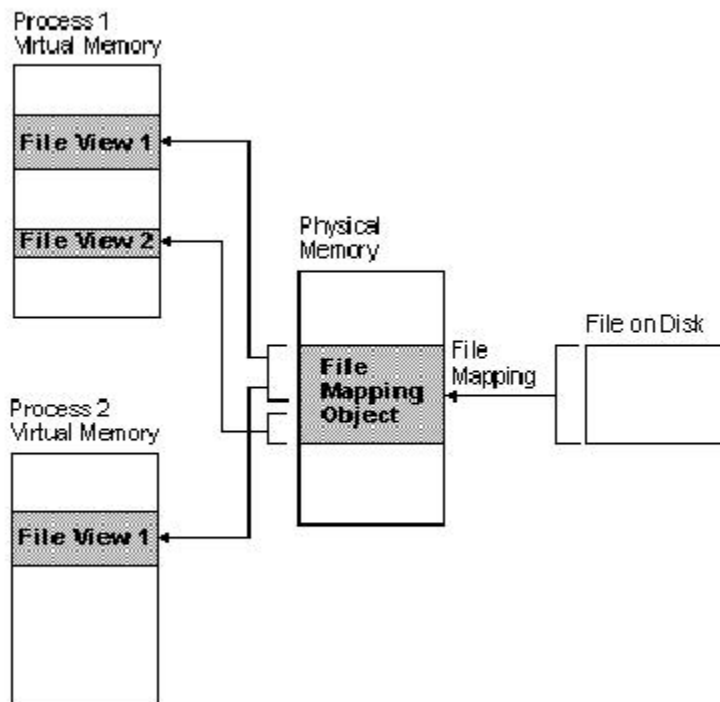
- Na podstawie IDT (interrupt dispatch table) uruchamia odpowiednie ISR (interrupt service routine), która obsługuje przerwanie
 - IRQL - poziom, priorytet przerwania
 - Tablice IDT pokazuje jakie ISR odpowiadają za dany IRQL
- Kernel dostarcza ISR dla wielu systemowych przerwań (zegarowe, związane z zasilaniem)
- Inne ISR dostarczane są przez sterowniki (zazwyczaj urządzeń)
 - Sterownik wiąże ISR z IRQL po przez utworzenie obiektu interrupt object i wysyłając go do kernela
 - Jądro łączy ten obiekt z odpowiednim wierszem tablicy IDT i gdy przerwanie wystąpi - uruchamia odpowiednią ISR dzięki temu obiektowi
 - Więcej niż jeden obiekt może być dołączony do danego poziomu IRQL w tablicy
- Przerwania z wyższym IRQL mają pierwszeństwo
- Można odroczyć wykonanie procedury za pomocą innego przerwania - DPC - deferredProcedure Calls
 - Wykorzystuje się to do wykonania kodu przy przejściu z wyższego poziomu IRQL na niższy



15. Przedstaw działanie i zastosowanie odwzorowania plików (file mapping). Podaj przykładowe wykorzystanie tego mechanizmu w praktyce.

- Część pliku lub jego całość jest mapowana na przestrzeń adresową procesu
- System tworzy file mapping obiekt w pamięci
 - Zawiera on część pliku z dysku
 - Gdy jest to potrzebne system odczytuje plik do obiektu lub obiekt zapisuje do pliku

- Proces tworzy obiekt File View na podstawie File Mapping Object aby odczytać/zapisać część pliku. FileView znajduje się w jego przestrzeni adresowej.
 - Rezultem pisania do File View jest zmiana w File Mapping Object
- File mapping używany jest do obsługi dużych plików i wymiany informacji pomiędzy procesami.
- Głównie chodzi o to, że plik mapowany jest na część pamięci operacyjnej przechowywanej na dysku (dzięki mechanizmowi pamięci wirtualnej). Ma on swój wpis w tablicy deskryptorów i odwoływanie się do niego przebiega na zasadzie stronicowania. Jeśli jakiś proces chce się do niego odwołać, to mechanizm stronicowania umieszcza część pamięci dyskowej z plikiem już jako część pamięci operacyjnej.
- W Linuksie używa się polecenie systemowego mmap() które robi w zasadzie to samo



16. W jaki sposób można doprowadzić do wycieków pamięci i jak można im zapobiec?

- Program nie zwalnia pamięci gdy nie jest ona już mu potrzebna
- Gdy program dodatkowo rezerwuje nową pamięć - zajmuje jej coraz więcej.
- Doprowadzić do tego może:
 - Nie zwalnianie pamięci dla nowo przydzielonej
 - Pozostawianie odwołań do nieużywanych obiektów w językach z Garbage Collectorem

- Nie zamykanie otwartych plików w programach
- Nie obsługiwanie wyjątków lub błędna ich obsługa
- Rzutowanie wskaźników
- polimorfizm - wykonanie złego destruktoru

```
BaseClass *x = new DerivedClass();  
delete x; (tylko jeśli nie ma zadeklarowanego destruktoru ~DerivedClass())
```
- zapobieganie:
 - Używanie języka z garbage collectorem
 - Przeciwdziałanie powyższym wypadkom
 Użycia odpowiednich narzędzi np valgrind do wykrywania wycieków

17. W jaki sposób zwalniana jest nieużywana już pamięć w maszynie wirtualnej ? Podaj przykładowe algorytmy działania (2).

- Obiekty, do których nie istnieją w programie żadne odwołania (referencje), są usuwane przez moduł czyszczenia pamięci - zbierania nieużytków, odświeżania (garbagecollector) - implementowany jako osobny wątek sterowania o niskim priorytecie.
 - **Zliczanie**, usuwanie obiektów, dla których ilość referencji wynosi zero (z takiego obiektu nie skorzystamy bo nie mamy się do niego jak odwołać, więc nie ma sensu trzymać go w pamięci)
 - **Budowanie drzewa referencji**, odnajdywanie trójkącików (referencja A wskazuje na B, B wskazuje na C ale C znowu wskazuje na A), lub dłuższych łańcuchów odwołujących się tylko do siebie
- Mechanizm odświeżania w Javie zwalnia programistę z obowiązku dealokacji obiektów poprzez jawne wywoływanie metod-destruktorów.

18. Opisz proces ładowania sterownika WDM do pamięci RAM podczas rozruchu systemu.

- Nie mam pojęcia czym to się różni od ładowania sterownika PnP z pytania 36.
 - Różni się to pierwszą częścią ładowania sterownika, związaną z odpytywaniem sprzętu przez magistralę by załadować sterownik.

19. Przetwórz klasyfikację sterowników w systemie Windows XP i podaj ich charakterystyki.

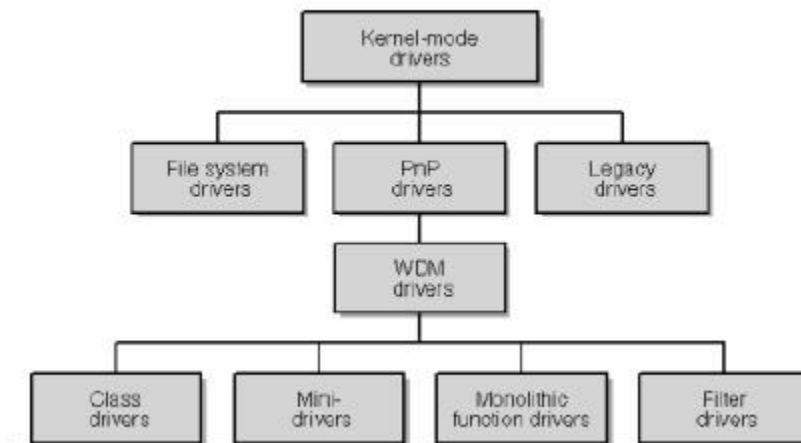
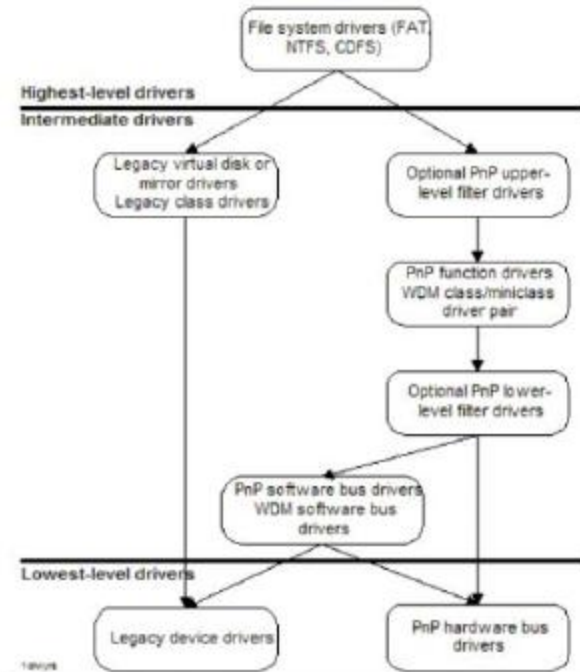
- **Sterowniki WDM** (Windows Driver Model) to sterowniki PnP, które dodatkowo implementują protokoły zarządzania energią i są zgodne zarówno z Windows 98/ME i Windows 2K/XP. To definicja. W praktyce sterowniki WDM nie zawsze oferują systemowi operacyjnemu wszystkie funkcje
- **Sterowniki klas**: łączą wspólną funkcjonalność dobrze zdefiniowanych klas urządzeń (np. kart sieciowych - NDIS class drivers, USB, napędów cdrom). Producenci sprzętu w

większości nie piszą własnych sterowników klas, ale używają jednego z dostarczonych przez Microsoft.

*"In [computing](#), a **class driver** is a type of hardware [device driver](#) that can operate a large number of different devices of a broadly similar type.(...)*

In technical terms, a class driver is used as a base or ancestor class for specific drivers which need to have slightly different or extended functionality, but which can take advantage of the majority of the functionality provided by the class driver. This concept is a key aspect of [object oriented programming](#), which when extended to drivers makes it much easier for hardware vendors to provide driver support for their products."

- **Minidriver:** Pisane przez producentów jako wsparcie dla sterowników klas.
*"A **miniport** is a type of hardware-driver, part of the [Windows Driver Model](#). These are [USB](#), Audio, [SCSI](#) and [network card](#) adapters. They should usually be source and binary compatible between [Windows 98](#) and [Windows 2000](#) and are hardware specific but control access to the hardware through a specific bus class driver."*
- **Sterowniki Filtrów** (filter driver): Dodają i/lub modyfikują zachowanie urządzenia lub wielu urządzeń
"[Filter drivers](#) are optional drivers that add value to or modify the behavior of a device and may be non-device drivers. A [filter driver](#) can also service one or more devices. Upper level filter drivers sit above the primary driver for the device (the function driver), while lower level filter drivers sit below the function driver and above the bus driver."
 - **Driver service:** *This is a type of kernel-level filter driver implemented as a [Windows service](#) that enables applications to work with devices."*
- **Sterowniki monolityczne** (Monolithic function driver): Jedne sterownik dla całego urządzenia (systemy typu embedded).
- **Sterowniki systemu plików:** implementują system plików dla dysku lokalnego lub podłączonego sieciowo.
- **Sterowniki typu Legacy:** Kontrolują sprzęt bez żadnego wsparcia od strony systemu operacyjnego (Sterowniki dla nieistniejącego sprzętu są dodane do systemu właśnie jako legacy drivers)
- **Sterowniki Pnp:** Implementują protokół Plug and Play.
- The **User-Mode Driver Framework** is a device-driver development platform first introduced with [Microsoft's Windows Vista operating system](#), and is also available for [Windows XP](#). It facilitates the creation of drivers for certain classes of devices.
- **Protocol drivers** implement a networking protocol such as TCP/IP, NetBEUI, and IPX/SPX.



20. Sposoby rejestracji sterownika w systemie Windows.

- ZwSetSystemInformation()
 - Pozyskujemy adres funkcji z ntdll.dll za pomocą GetProcAddress
 - Nie możesz zwolnić sterownika ani usunąć jego pliku aż do restartu
- Wykorzystując Service Control Manager - **CreateService** API
 - Sc.exe create nazwa binpath=ścieżka parametry

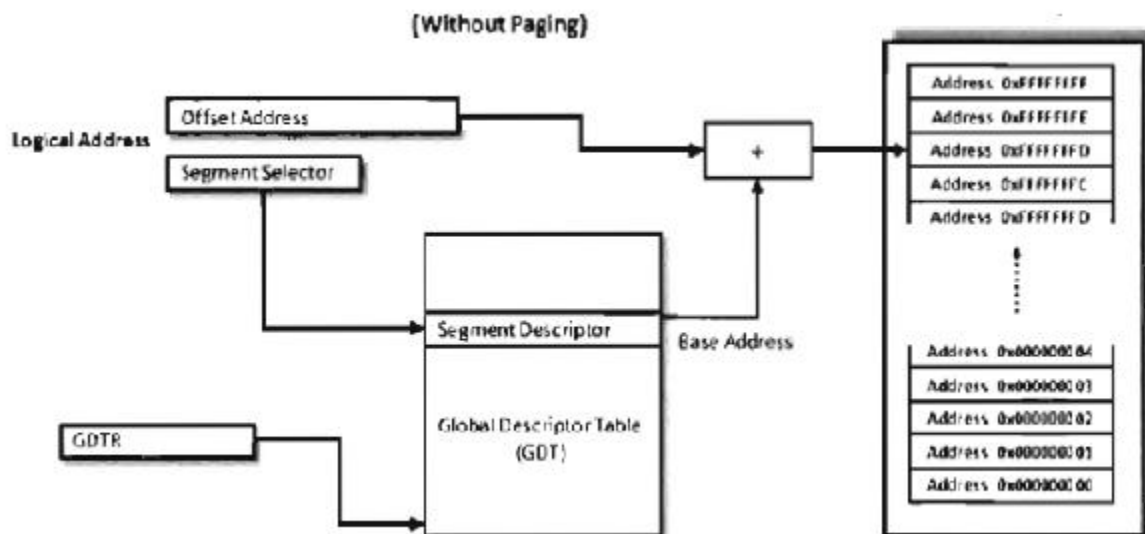
- Zapis do \Device\PhysicalMemory

21. Omówić implementację pamięci wirtualnej za pomocą segmentacji.

22. Jakie zalety ma realizacja pamięci wirtualnej za pomocą stronicowania w porównaniu z realizacją za pomocą segmentacji?

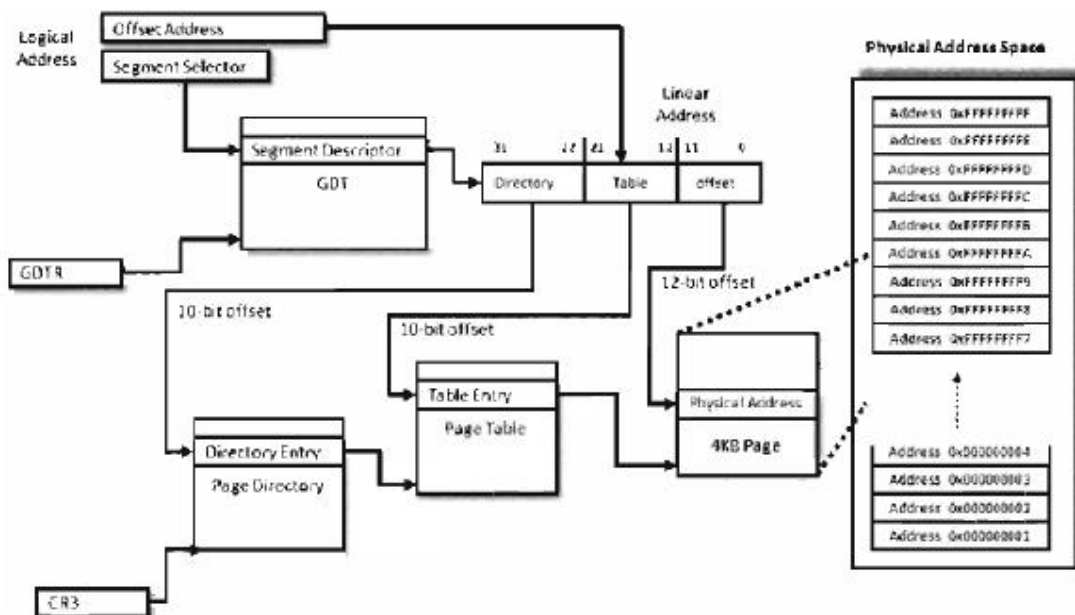
24. Opisz mechanizm wywołania funkcji systemowej na poziomie procesora (rejestr GDT)

- Segmentowy model pamięci
 - Dzielimy pamięć na segmenty (każdy ma swój numer)
 - Adres: SELEKTOR (numer wskazujący segment) | PRZESUNIĘCIE (w danym segmencie)
 - Segmenty mogą mieć różną wielkość
- Pamięć, a tryby pracy procesora
 - Tryb rzeczywisty
 - Oparty o segmenty
 - $\text{NUMER SEGMENTU (16 bitowy deskryptor)} \times 16 + \text{OFFSET (16 bitowy)} = \text{ADRES FIZYCZNY}$
 - Możliwość zaadresowania 1 MB
 - Pamięć jest uporządkowana
 - Każda aplikacja ma dostęp do całej pamięci (m. in. Wektorów przerwań i innych programów)
 - Można modyfikować adresy f-cji systemowych
 - Tryb chroniony z segmentacją
 - Procesy mają dostęp tylko do przydzielonych im segmentom

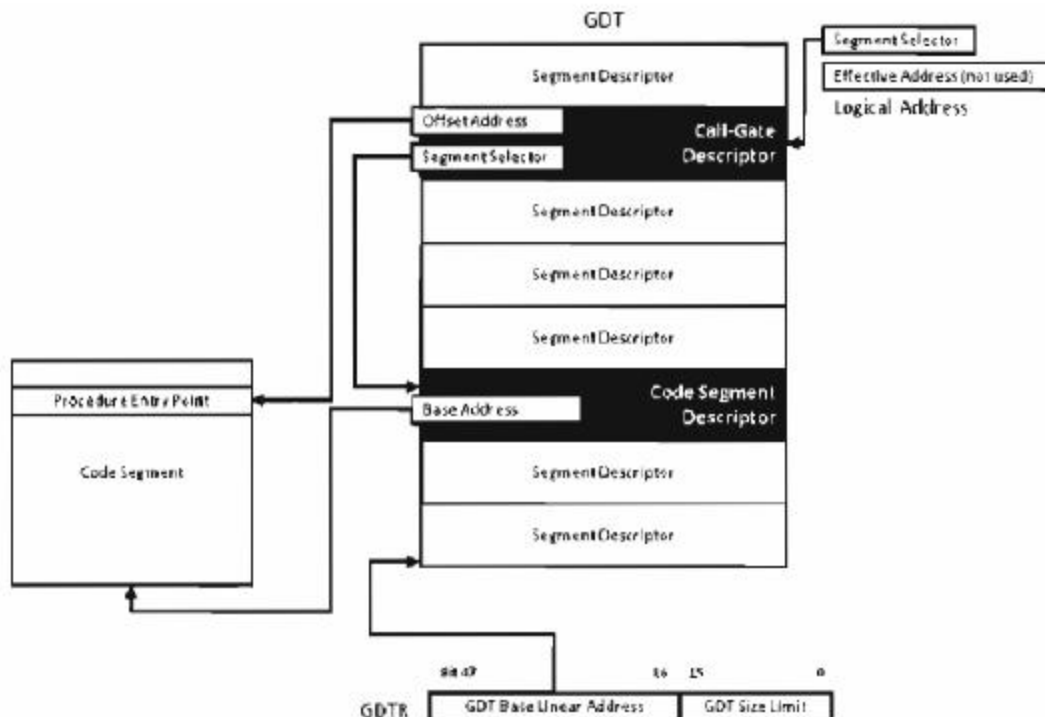


-
- Rejestr GDTR - adres początku tablicy deskryptorów

- Selektor wskazuje na deskryptor. Z niego pobierany jest adres segmentu. Po powiększeniu go od OFFSET mamy ADRES FIZYCZNY
- SELEKTOR - w nim jest wskaźnik na deskryptor i poziom uprzywilejowania segmentu wywołującego
 - [Segment selectors](#) contain a field called the requested privilege level (RPL). This allows a program to request a resource at a lower privilege level than it would otherwise use.
- DESKRYPTOR - w nim jest wskaźnik na segment, jego rozmiar oraz numer DPL (*Descriptor Privilege Level*)
 - DPL - wywołujący segment musi mieć poziom uprzywilejowania $DPL \geq CPL$ (Current Privilege Level)
 - Zawiera bajt dostępu, który pokazuje czy strona znajduje się na dysku i przy "obróbce" trzeba ją wczytać do pamięci operacyjnej (bit P)
- Stronicowanie
 - Adres z deskryptora jest dzielony na 3 części
 - Numer katalogu (10 bitów)
 - Numer tablicy (10 bitów)
 - Offset (12 bitów)



- CR3 - wskazuje na tablice katalogów stron
- Poziomy uprzywilejowania, dostanie się do niższego poziomu



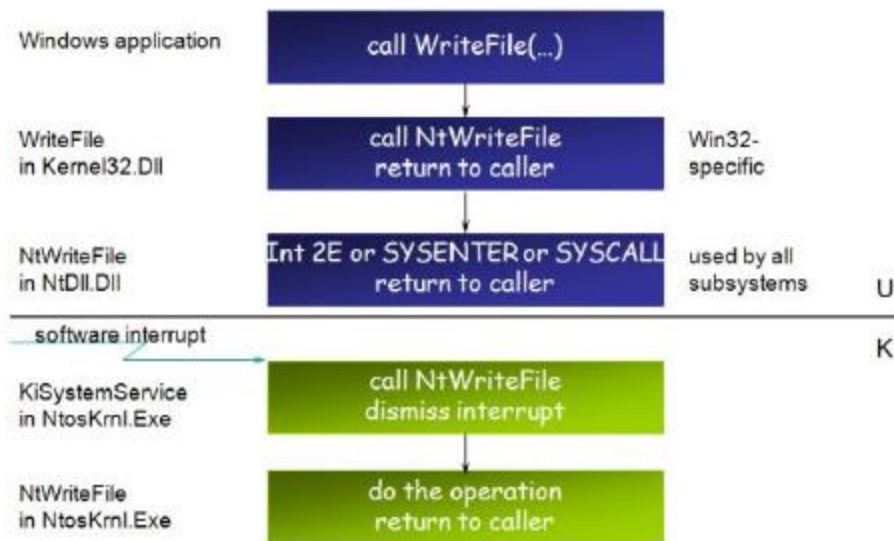
- Wśród deskryptorów w tablicy znajdują się także bramy
- Brama ma w sobie selektor na inny deskryptor z niższym poziomem uprzywilejowania
- ADRES z 2 deskryptora | OFFSET z selektora wskazującego na bramę = ADRES PROCEDURY SYSTEMOWEJ
- Każde zadanie ma 3 stosy - dla każdego poziomu uprzywilejowania
- Przerwania działają tak samo jak bramy
- Zalety stronicowania
 - Brak fragmentacji zewnętrznej
 - W pamięci robią się dziury z których nikt już nie skorzysta
 - Strony są stałej wielkości i każda może być użyta
 - **Separacja przestrzeni adresowej dla każdego procesu = bezpieczeństwo**
 - Odzworowanie pliku dyskowego w przestrzeń adresową procesu
 - **Powiększenie pamięci fizycznej**
- wywoływanie podprogramów na wyższych poziomach uprzywilejowania możliwe jest tylko za pośrednictwem specjalnych deskryptorów zwanych furtkami wołania (ang. call gates) – furtki wołania, przechowywane w tablicy deskryptorów, stanowią jedyne legalne punkty wejścia do podprogramów;
- każda furtka ma przypisany pewien poziom uprzywilejowania; furtka wołania jest dostępna dla wszystkich programów uprzywilejowanych nie gorzej niż poziom furtki; wywołanie podprogramu za pośrednictwem furtki wołania polega na wykonaniu rozkazu CALL typu FAR: pole selektora tego rozkazu wskazuje furtkę, a offset jest ignorowany;
- pole DPL furtki określa konieczny stopień uprzywilejowania dla programu, który korzysta z furtki; zatem furtka może być użyta tylko wówczas, gdy $DPL \geq CPL$, gdzie CPL oznacza bieżący poziom uprzywilejowania;

23. W jaki sposób w architekturze IA-32 została zrealizowana koncepcja rejestru bazowego i granicznego jako metody izolacji procesów?

- Aby zapewnić poprawne działanie systemu, należy chronić wektor przerwania oraz same procedury obsługi tych przerwania
- Ochronę tę można uzyskać za pomocą 2 rejestrów: bazowego i granicznego
 - Bazowy przechowuje najmniejszy dopuszczalny adres
 - Graniczny zawiera rozmiar obszaru pamięci
- Procesy mają przydzielone oddzielne segmenty pamięci na kody programów oraz na dane. Z każdym segmentem związane są dwa rejestry: rejestr bazowy oraz rejestr graniczny.
- Część adresowa instrukcji pobranej do wykonania z programu użytkownika musi mieścić się w przedziale wyznaczonym przez zawartości tych dwóch rejestrów.
- W IA-32 w deskryptorze znajduje się adres początkowy (traktujemy to jako rejestr bazowy) i rozmiar (można obliczyć graniczny). Dzięki temu jest sprawdzane czy wołany adres nie wychodzi poza zakres.

25. Opisz mechanizm wywołania funkcji systemowej na poziomie jądra systemu operacyjnego Windows XP

- Wywołanie funkcji Windowsowej np. WriteFile. Przez systemowe biblioteki dochodzi to do NTDll
- NTDll wykonuje funkcję Int2E, SYSENTER lub SYSCALL, która powoduje tzw. "przerwanie programowe"
- Systemowy dispatcher obsługuje przerwanie
 - Szuka odpowiedniego wpisu w tablicy przerwania
 - Wpis zawiera numer tablicy i numer indeksu w tej tablicy - tam znajduje się API funkcji systemowej
 - Po takim odwołaniu funkcja zostaje uruchomiona



26. Wyjaśnić na czym polegają różnice w sposobie wykonywania funkcji blokujących, nieblokujących i asynchronicznych.

- Funkcje działające na gniazdkach blokujących powodują "zamrożenie" wykonywania programu aż do wykonania całej operacji, np. odebrania datagramu. Używanie takich funkcji wymaga specjalnej uwagi w przypadkach, gdy operacje wykonywane są na kilku gniazdkach: proces nie może rozpocząć operacji czytania z gniazdka, gdyż w trakcie oczekiwania na dane z jednego gniazdka, w innym mogą pojawić się dane.
- Z kolei funkcje działające na gniazdkach nieblokujących zwracają od razu odpowiedź (poprawnie. Błąd, nic się nie wydarzyło).
- W systemie Windows stosuje się często gniazdko asynchroniczne, przy których wykonywanie funkcji zostaje od razu zakończone, ale jednocześnie określa się komunikat, który zostanie wysłany do funkcji okienkowej, gdy wystąpi oczekiwane zdarzenie.
- Te funkcje niekoniecznie muszą "być gniazdkowe" (w materiałach po prostu było omówione na gniazdkach), to taki typ funkcji występujących także w innych zastosowaniach

27. Przedstaw działanie procesu planisty w systemie Windows XP.

- Windows XP przydziela czas na poziomie wątków
- Planista bazuje na wyłłaszczaniu w oparciu o priorytety, najpierw wykonywane są wątki o najwyższych priorytetach (round robin z priorytetami)
- 32 poziomy priorytetów
 - szesnaście poziomów czasu rzeczywistego (16-31)
 - piętnaście poziomów zmiennych (1-15)

- jeden poziom systemowy (0) zarezerwowany dla wątku podstawowego
- Wątek dziedziczy priorytet procesu
- System może (tymczasowo) zwiększyć priorytet wątku
 - Po zakończeniu operacji I/O
 - Po odczekaniu na zdarzenie
 - Wątki GUI - aktywność okienka
 - Proces nie działa od dłuższego czasu
- Po upływie kwantu (jednostkowego czasu działania wątku) system sprawdza czy inny wątek o tym samym lub wyższym priorytecie nie czeka na uruchomienie lub czy nie należy obniżyć priorytetu wątku
- Wątek może zostać wywłaszczony przed upływem wyznaczonego czasu przez wątek o wyższym priorytecie i odroczone (mechanizm DPC)

28. Wymień i scharakteryzuj metody łączenia kodu bibliotecznego z kodem programu.

- Biblioteki statyczne
 - łączona jest z programem w momencie konsolidacji
 - Linkowanie statyczne polega na dołączeniu odpowiedniego kodu funkcji do programu na etapie kompilacji.
 - Eliminuje problem DLL hell
- Konsolidacja dynamiczna w czasie ładowania (LTL)
 - Linkowanie dynamiczne polega na dołączaniu do programu jedynie odpowiednich odwołań do funkcji bibliotecznych
 - Sam kod funkcji nie jest dołączany do programu na etapie kompilacji.
 - Przy uruchomieniu programu, biblioteka jest natomiast ładowana do pamięci i udostępnia odpowiednie funkcje pracującemu programowi.
 - jeden egzemplarz biblioteki w pamięci dla wszystkich programów które z niej korzystają
- Konsolidacja dynamiczna w czasie wykonywania (RTL)
 - Ładowanie i zwalnianie biblioteki prosto w kodzie programu (LoadLibrary(), FreeLibrary())
 - Dzięki GetProcAddress() dostajemy adres funkcji bibliotecznej

29. Opisz budowę pliku PE

- Portable Executable (PE) - format plików wykonywalnych (.exe), obiektowych oraz dynamicznych bibliotek (.dll)
- Określenie "portable" (pol. "przenośny") odnosi się do uniwersalności formatu na wielu architekturach systemów komputerowych
- Format PE jest po prostu strukturą danych, która enkapsuluje informacje potrzebne systemowi do zarządzania kodem wykonywalnym.

- DOS Stub + sygnatura
 - Sygnatura PE
 - Wykonywalny program DOS
 - struktura opisująca część DOSową pliku PE
- Nagłówek pliku
 - Typ procesora
 - Liczba sekcji
 - Czasy utworzenia
 - Parametry tablicy symboli
 - Rodzaj pliku (.exe, .dll)
- Opcjonalny nagłówek
 - Rozmiary segmentów danych, kodu
 - Adres Punktu wejściowy
- Katalogi danych
 - Wskazują na specyficzne struktury
 - mała struktura opisująca inne struktury używane w pliku
- Nagłówki sekcji
 - struktura opisująca pojedynczą sekcję
- Sekcje danych

BSS is only used to store uninitialized data
typically the .text section (which holds program code) is mapped as execute/readonly, and the .data section (holding global variables) is mapped as no-execute/readwrite.
- Tak wygląda plik:

PE File Format

MS-DOS MZ Header
MS-DOS Real-Mode Stub Program
PE File Signature
PE File Header
PE File Optional Header
.text Section Header
.bss Section Header
.rdata Section Header
. . .
.debug Section Header
.text section
.bss Section
.rdata Section
. . .
.debug section

"A PE file consists of a number of headers and sections that tell the *dynamic linker* how to map the file into memory. An executable image consists of several different regions, each of which require different memory protection; so the start of each section must be aligned to a page boundary. For instance, typically the .text section (which holds program code) is mapped as execute/readonly, and the .data section (holding global variables) is mapped as no-execute/readwrite. However, to avoid wasting space, the different sections are not page aligned on disk. Part of the job of the dynamic linker is to map each section to memory individually and assign the correct permissions to the resulting regions, according to the instructions found in the headers.

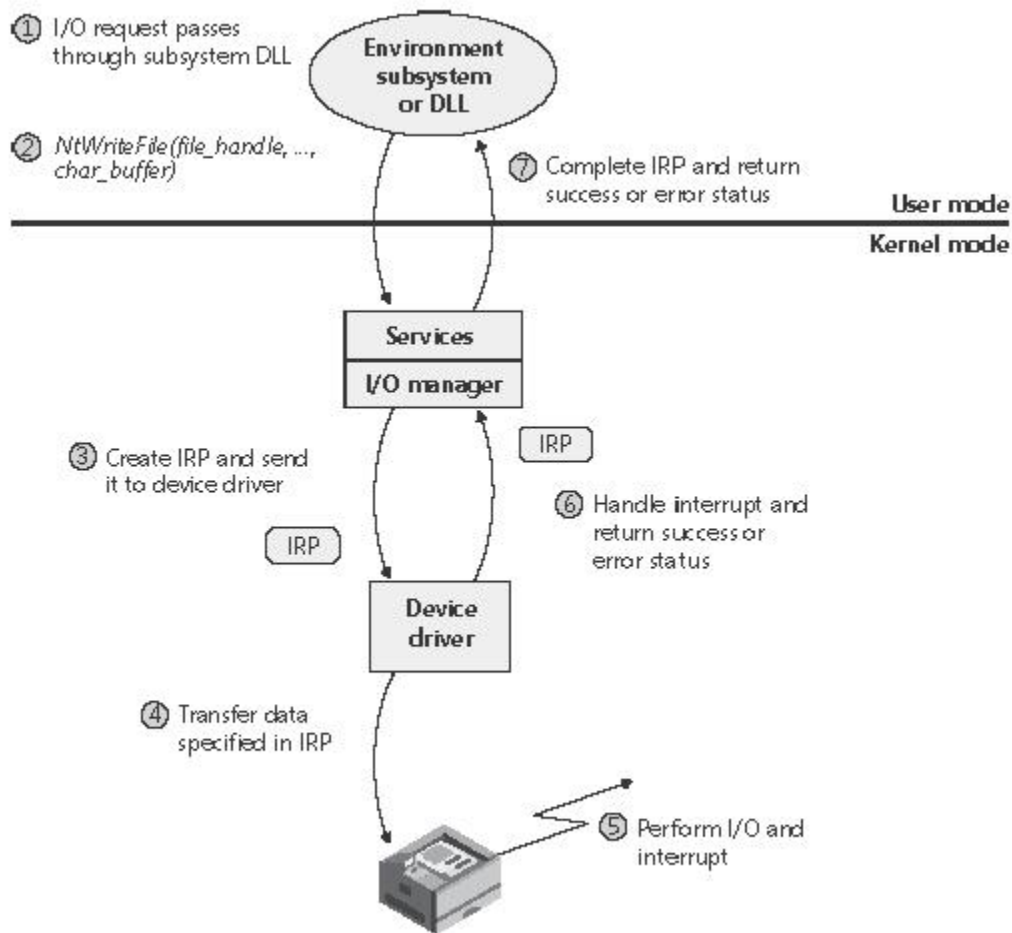
PE files do **not contain position-independent code**. Instead they are compiled to a preferred base address, and all addresses emitted by the compiler/linker are fixed ahead of time. If a PE file cannot be loaded at its preferred address (because it's already taken by something else), the operating system will rebase it. This involves recalculating every absolute address and modifying the code to use the new values. It also slows down loading of the module significantly. For this reason rebasing is to be avoided wherever possible, and the DLLs shipped by Microsoft have base addresses pre-computed so as not to overlap. In the no rebase case PE therefore has the advantage of very efficient code, but in the presence of rebasing the memory usage hit can be expensive. **Contrast this with ELF which uses fully position independent code and a global offset table**, which trades off execution time against memory usage in favor of the latter."

30. Przedstaw kolejne kroki w komunikacji pomiędzy aplikacją użytkową a sprzętem w systemie Windows

- Skupić się na tym rysunku z komunikacji synchronicznej
- ogólnie
 - User-mode API
 - Wykonanie odpowiedniej funkcji w języku programowania
 - API usług systemu IO
 - Biblioteka NT wpisuje odpowiednie strumienie, będące zapytaniem, do wirtualnych plików, reprezentujących dane urządzenie
 - I/O manager
 - Dynamicznie dostarcza zapytania z warstwy użytkownika do odpowiedniego sterownika urządzenia
 - Procedury obsługi sterowników
 - Abstrakcja HAL
- Synchronicznie
 - Aplikacja uruchamia odpowiednią funkcję potrzebującą komunikacji ze sprzętem
 - Podsystem DLL (ntDLL) wysyła zapytanie IO
 - IO manager tworzy pakiet IRP opisujący to zapytanie i wysyła go do sterownika
 - Sterownik wysyła dane z IRP (I/O request packet) do urządzenia
 - Sterownik sygnalizuje przerwaniem, że to już wszystkie dane
 - Urządzenie wykonuje żadaną operację i na końcu robi przerwanie
 - Przerwanie obsługiwane jest w ten sposób, że ISR tylko odczytuje status urządzenia i kolejkuje DPC (deferred procedure call) (coś co

będzie zrobione w czasie wolnym od normalnych przerw), które zrobią resztę

- Sterownik wysyła przerwanie
- Sterownik woła funkcję IO managera świadczącą o zakończeniu przetwarzania
- IO manager kończy obsługę zapytania
 - Trzeba wysłać status lub odpowiedź
 - IO manager kopiuje dane do przestrzeni adresowej procesu



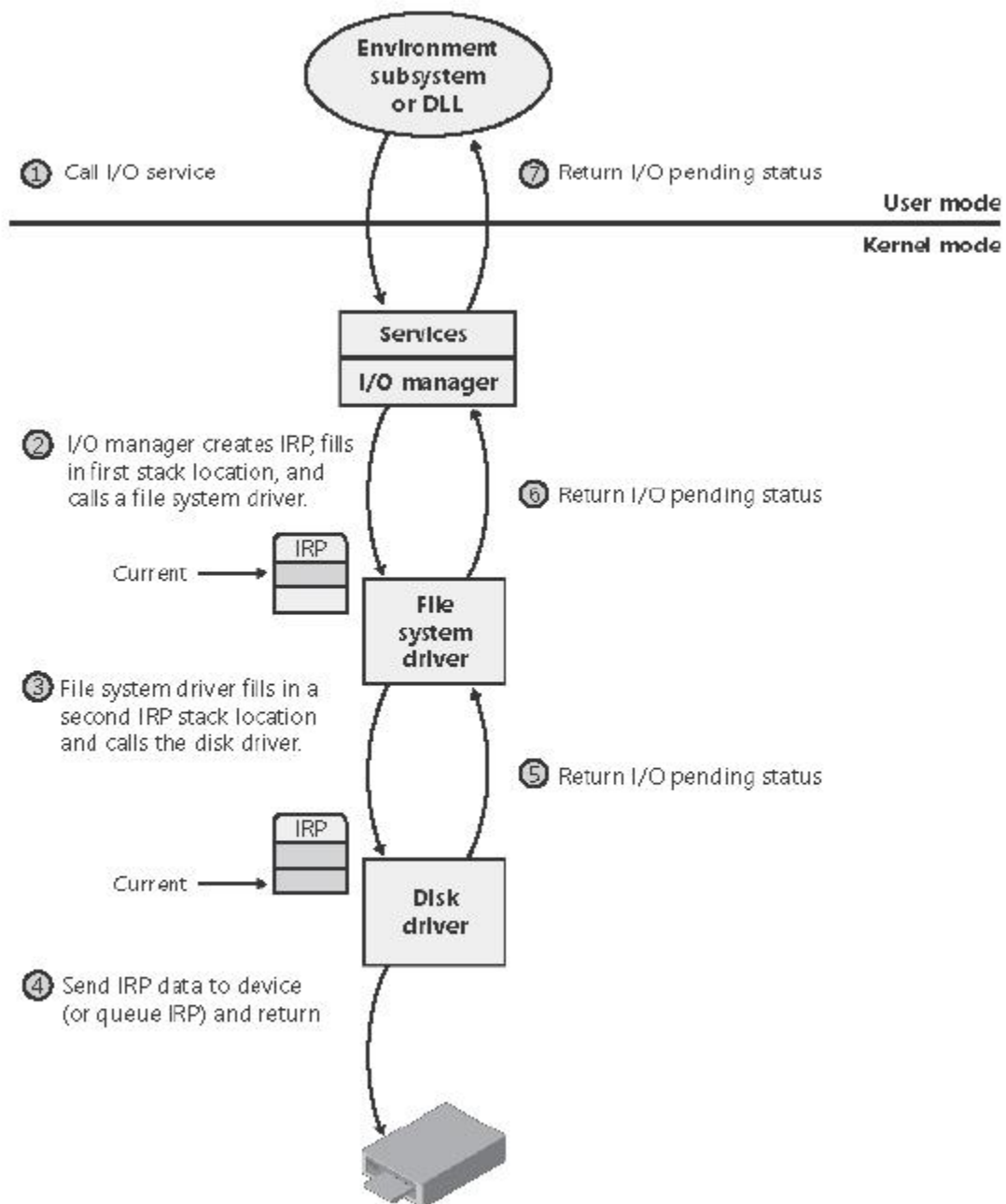
- **Asynchronicznie**

- IO manager odwleka koniec obsługi zapytania, aż procesor znajdzie się w "kontekście" procesu, który wywoływał dane zapytanie
- Robi to po przez przerwania APC, które są wywołane gdy dany wątek ponownie jest wykonywany
- Odwlekanie spowodowane jest tym, że IO manager nie może zna przestrzeni adresowej wątku wołającego zapytanie (w synchronicznym zna)

- **Wielowarstwowe**

Część dosowa

Część PE



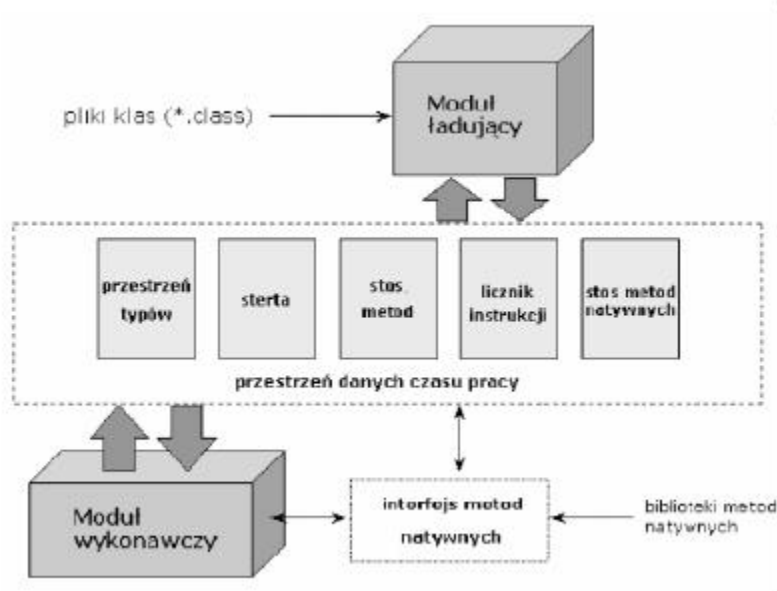
31. Wyjaśnić na czym polega komunikacja między procesami za pomocą potoków nazwanych.

- Potoki nazwane określane też są terminem kolejek FIFO albo nazwanych łącz komunikacyjnych.
- Potok nazwany (lub łącze nazwane) jest to rozwinięcie idei potoku, polegające na stworzeniu specjalnego pliku, który ma służyć jako łącznik między procesami. Zapewnia to wygodny mechanizm przekazywania danych pomiędzy niespokrewnionymi ze sobą procesami.
- Kolejki FIFO zachowują się podobnie jak potoki nienazwane utworzone za pomocą funkcji pipe. Jeden z procesów otwiera plik FIFO do zapisu, zaś drugi do odczytu, po czym dane mogą przepływać przez kolejkę jak przez zwykły potok nienazwany.
- Potoki nienazwane mogą być stosowane tylko między procesami "spokrewnionymi", natomiast potoki nazwane mogą być łączem pomiędzy potokami zupełnie niezależnymi. Jest tak dlatego, że są one pewnego rodzaju plikiem i mają swoją nazwę, przez którą można się do nich odwołać. Natomiast sam przepływ danych wygląda tak samo jak w przypadku potoków nienazwanych.
- Mogą do nich zapisywać lub czytać jeden lub więcej procesów.

32. Omówić sposób wykonania programu w środowisku maszyny wirtualnej JVM

- Moduł ładowania
 - Wyszukiwanie i pozyskiwaniu plików .class
 - Ładowanie i zarządzanie pamięcią dla poszczególnych klas
 - Tworzenie struktur systemowych: rejestru PC, stosu, sterty, obszaru metod
- Część wykonawcza
 - Weryfikator
 - Przed rozpoczęciem wykonywania załadowanego kodu bajtowego jest on sprawdzany
 - Weryfikacja pliku klasy: Weryfikacja sygnatury i struktury pliku klasy
 - Weryfikacja typu systemu: tworzenia podklas, stałych, wskaźników, obszarów pamięci
 - Weryfikacja kodu bajtowego: analiza metod, kontrola argumentów, inicjalizacji zmiennych, stosu argumentów, operacji arytmetycznych (wystąpienie nadmiaru i niedomiaru)
 - Weryfikacja podczas wykonania typu danych oraz dostępu doń
 - Wykonanie JIT - just in time
 - Kompilacja kodu maszynowego w locie, bezpośrednio przed wykonaniem danego fragmentu
 - Maszyna wirtualna decyduje co kompilować, a co interpretować. Zazwyczaj kompiluje to co wiele razy zostało już uruchomione
 - Moduł bezpieczeństwa
 - Nadzoruje politykę dostępu klasy do zasobów lokalnych i sieciowych
 - Menadżer bezpieczeństwa musi wyrazić zgodę na dostęp do krytycznych zasobów (bibliotek, procesów operacyjnych, systemu plików itd..)
 - Moduł czyszczenia pamięci

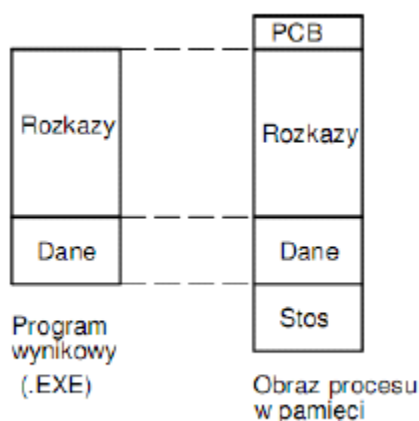
- Zwalnia programistę z obowiązku dealokacji obiektów
- Czesciow zapobiega wyciekom pamieci



33. Opisz proces ładowania programów do pamięci operacyjnej

- Polega na przydzieleniu odpowiednio dużego obszaru pamięci, przeznaczonego na: blok PCB (*Process Control Block*), rozkazy, dane i stos, a następnie przepisanie kodu danych i danych programu z pliku dyskowego do pamięci operacyjnej, trzy podstawowe techniki
- w przypadku **ładowania bezwzględnego** cały kod programu musi być w pełni określony w chwili ładowania go do pamięci; programista może posługiwać się adresami fizycznymi lokacji pamięci, lub też adresy te mogą wyznaczone przez kompilator (assembler) i linker; niezbędna jest jednak znajomość adresu początkowego obszaru pamięci, do którego zostanie wpisany program;
- Chodziło tu o to jak program z pliku PE jest ładowany do pamięci. Pan Dziubich powiedział, że jest to rodzaj ładowania bezwzględnego - mieliśmy to rozwinąć. Poszukałem trochę i nie wiem czy do końca o to chodzi, ale znalazłem coś takiego:
 - firstly check if the file is being run in DOS mode, if it is then it displays the error "This file cannot be run in DOS mode". If not, then it maps each of the files sections then loads it into memory, and then executes it.
 - PE files do not contain position-independent code. Instead they are compiled to a preferred base address, and all addresses emitted by the compiler/linker are fixed ahead of time. If a PE file cannot be loaded at its preferred address (because it's already taken by something else), the operating system will rebase it. This involves recalculating every absolute address and modifying the code to use the new values. The loader does this by comparing the preferred

and actual load addresses, and calculating a delta value. This is then added to the preferred address to come up with the new address of the memory location. Base relocations are stored in a list and added, as needed, to an existing memory location. The resulting code is now private to the process and no longer shareable, so many of the memory saving benefits of DLLs are lost in this scenario. It also slows down loading of the module significantly. For this reason rebasing is to be avoided wherever possible, and the DLLs shipped by Microsoft have base addresses pre-computed so as not to overlap. In the no rebase case PE therefore has the advantage of very efficient code, but in the presence of rebasing the memory usage hit can be expensive. Contrast this with ELF which uses fully position independent code and a global offset table, which trades off execution time against memory usage in favor of the latter. (za angielską wiki)



34. Omów budowę podstawowego szkieletu sterownika urządzeń WDM w systemie Windows.

- An initialization routine The I/O manager executes a driver's initialization routine, which is typically named **DriverEntry**, when it loads the driver into the operating system. The routine fills in system data structures to register the rest of the driver's routines with the I/O manager and performs any global driver initialization that's necessary.
- An **add-device** routine A driver that supports Plug and Play implements an add-device routine. The PnP manager sends a driver notification via this routine whenever a device for which the driver is responsible is detected. In this routine, a driver typically allocates a device object (described later in this chapter) to represent the device.
- A set of **dispatch routines** **Dispatch** routines are the main functions that a device driver provides. Some examples are open, close, read, and write and any other capabilities the device, file system, or network supports. When called on to perform an I/O

operation, the I/O manager generates an IRP and calls a driver through one of the driver's dispatch routines.

35. Ile stosów ma procedura w trybie chronionym? Wyjaśnij dlaczego tak się dzieje.

Dana procedura wykonywana jest przez wątek. Każdy wątek posiada 2 stosy – po jednym w trybie jądra i użytkownika. Powodem tego jest bezpieczeństwo – by nic co działa w trybie użytkownika nie miało dostępu do danych z trybu jądra.

36. Opisz proces ładowania sterowników dla urządzeń PnP.

"1. The I/O manager invokes the driver entry routine of each boot-start driver. If a boot driver has child devices, the I/O manager enumerates those devices, reporting their presence to the PnP manager. The child devices are configured and started if their drivers are boot-start drivers. If a device has a driver that isn't a boot-start driver, the PnP manager creates a devnode for the device but doesn't start it or load its driver.

2. After the boot-start drivers are initialized, the PnP manager walks the device tree, loading the drivers for devnodes that weren't loaded in step 1 and starting their devices. As each device starts, the PnP manager enumerates its child devices, if it has any, starting those devices' drivers and performing enumeration of their children as required.

PnP manager loads the drivers for detected devices in this step regardless of the driver's Start value. (The one exception is if the Start value is set to disabled.) At the end of this step, all Plug and Play devices have their drivers loaded and are started, except devices that aren't enumerable and the children of those devices.

3. The PnP manager loads any drivers with a Start value of system-start that aren't yet loaded. Those drivers detect and report their nonenumerable devices. The PnP manager loads drivers for those devices until all enumerated devices are configured and started.

4. The service control manager loads drivers marked as auto-start."
z "Windows Internals 5th Ed."

- Menadżer Pnp woła menadżera pamięci w celu mapowania pamięci zanim sterownik zostanie załadowany.
- Memory manager woła DriverEntry.
- Menadżer Pnp woła funkcję AddDevice sterownika, i w ten sposób załadowany sterownik jest powiadomiony o wykryciu nowego urządzenia.
- Menadżer Pnp wysyła IRP do sterownika magistrali aby pozyskać informacje o potrzebnych zasobach
- Menadżer Pnp konfiguruje sprzęt i wysyła IRP_MJ_PNP do sterownika. Urządzenie jest uruchomione.

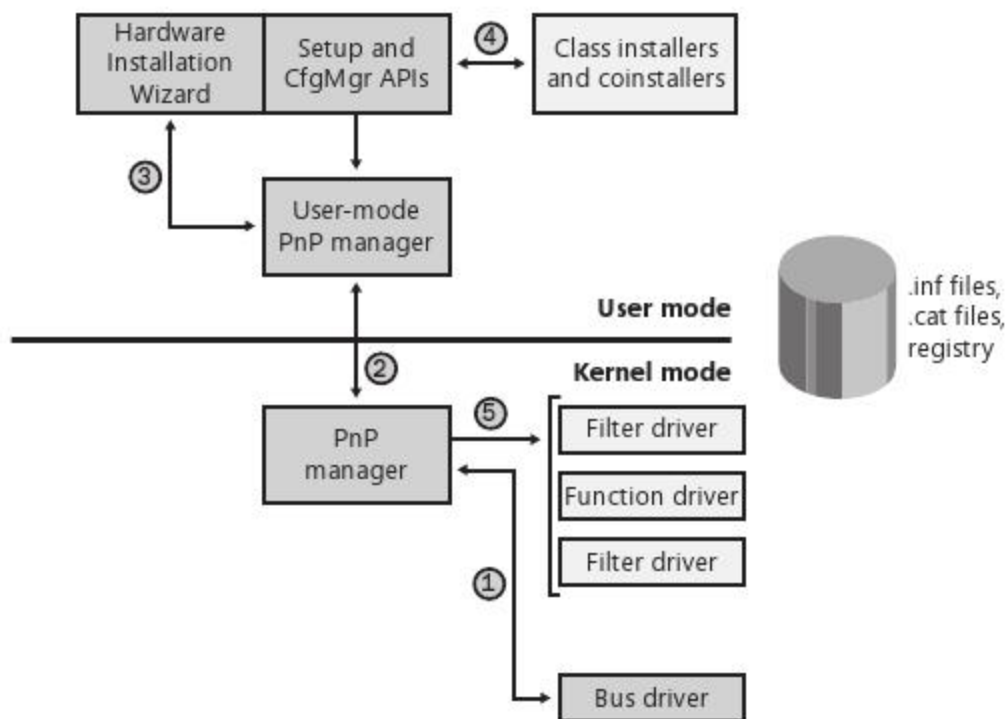
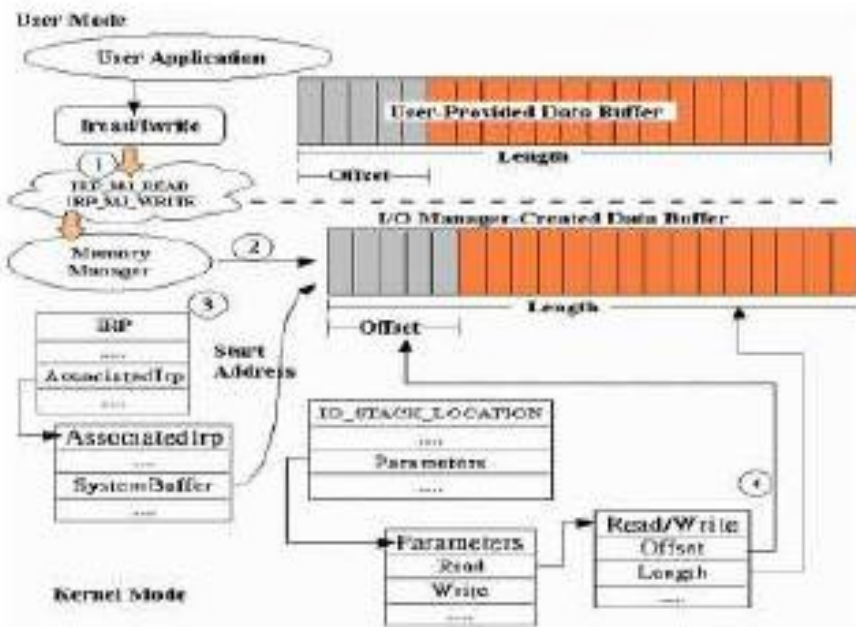


FIGURE 7-42 Driver installation components

37. Wymień i opisz sposoby komunikacji aplikacji użytkowej ze sterownikiem WDM.

- **Buforowany**

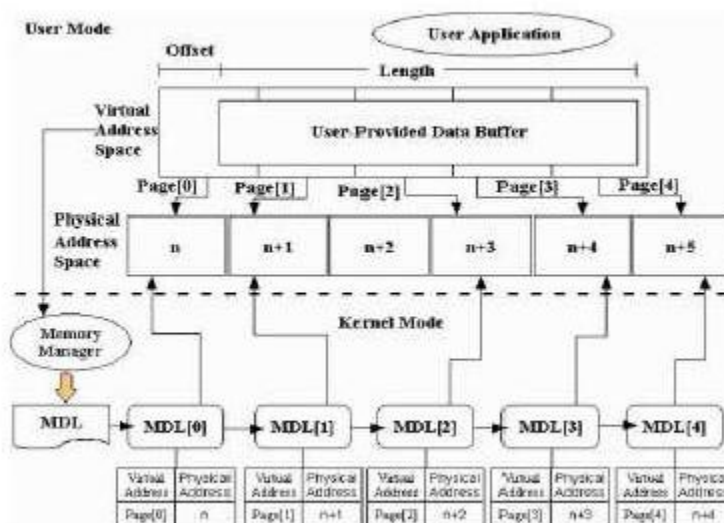
Buforowane operacje I/O



The operating system creates a nonpaged system buffer, equal in size to the application's buffer. For write operations, the I/O manager copies user data into the system buffer before calling the driver stack. For read operations, the I/O manager copies data from the system buffer into the application's buffer after the driver stack completes the requested operation.

- **Bezpośredni**

Direct I/O



- The operating system locks the application's buffer in memory. It then creates a memory descriptor list (MDL) that identifies the locked memory pages, and passes the MDL to the driver stack. Drivers access the locked pages through the MDL.
- **Własny (neither)**

38. Omów typowe działanie systemu operacyjnego przy błędach strony (page fault handling).

When a page fault occurs, the hardware cannot do anything else with the instruction that caused the page fault and thus it must transfer control to an operating system routine (this is the page fault handler). The page fault handler must then decide how to handle the page fault. It can do one of two things:

- It can decide the virtual address is just simply not valid. In this case, Windows will report this error back by indicating an exception has occurred (typically STATUS_ACCESS_VIOLATION)
- It can decide the virtual address is valid. In this case, Windows will find an available physical page, place the correct data in that page, update the virtual-to-physical page translation mechanism and then tell the hardware to retry the operation. When the hardware retries the operation it will find the page translation and continue operations as if nothing had actually happened.

Hardware generates a page fault for page accesses where:

- the page corresponding to the requested address is not loaded in memory.
- the page corresponding to the memory address accessed is loaded, but its present status is not updated in hardware.

The closely related exception known as the **protection fault** is generated for page accesses where:

- the page is not part of the program, and so is not mapped in program memory.
- the program does not have sufficient privileges to read or write the page.
- the page access is legal, but it is mapped with [demand paging](#).

Protection fault can also be generated for many other invalid accesses not related to paging.

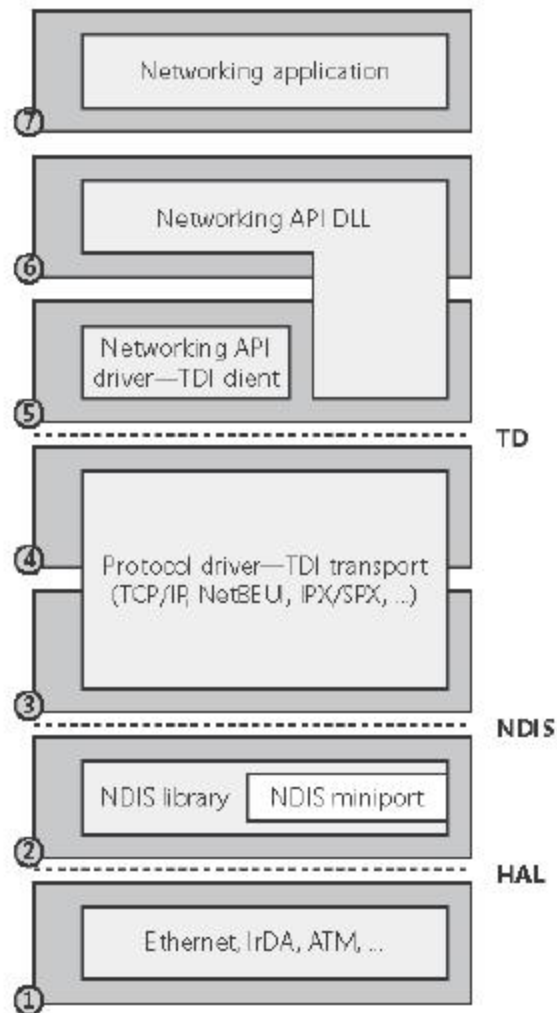
A *hard page fault* must be resolved by reading page contents from the page's *backing store*, which is either the system paging file or a memory-mapped file created by the process.

A *soft page fault* can be resolved without accessing the backing store. A soft page fault occurs when:

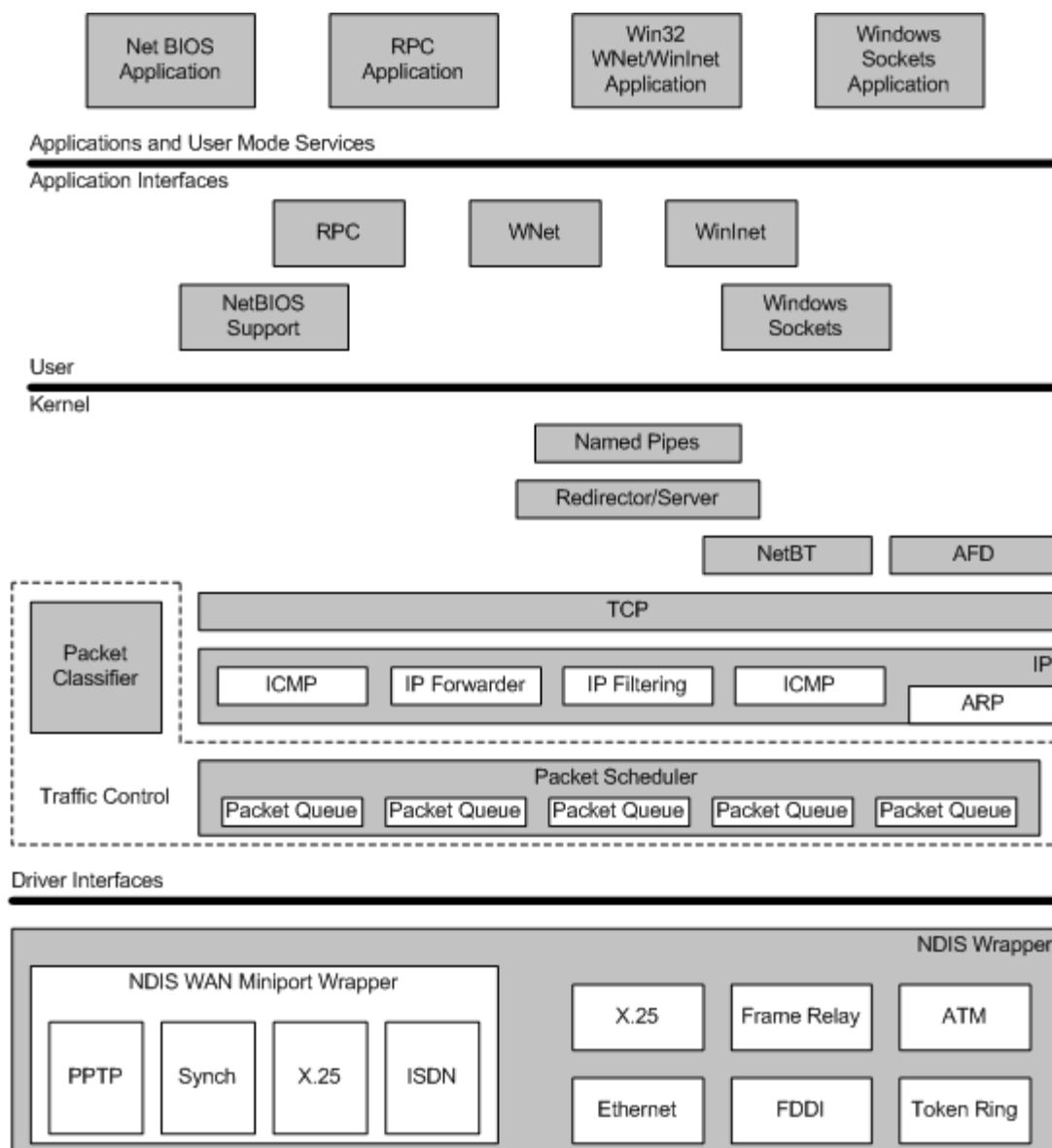
- The page is in the working set of some other process, so it is already resident in memory.
- The page is in transition, because it either has been removed from the working sets of all processes that were using the page and has not yet been repurposed, or it is already resident as a result of a memory manager prefetch operation.
- A process references an allocated virtual page for the first time (sometimes called a *demand-zero fault*).

39. Opisz architekturę stosu sieciowego w Windows

Figure 13-2. OSI model and Windows **networking** components



Schemat dla TCP/IP:



Stos protokołów w systemie Windows ułożony jest warstwowo. Jest tak by uniezależnić protokoły warstw wyższych od implementacji tych niższych. Poszczególne warstwy są oddzielone czymś w rodzaju 'warstwy buforującej'. Najniższy poziom stosu sieciowego stanowi warstwa HAL – sprzętowa. Mogą to być karty sieciowe lub inne urządzenia służące do przesyłu danych w sieci.

Wyżej znajduje się biblioteka NDIS. Służy ona inkapsulacji sterowników, ukrywa przed nimi środowisko Windowsa. Zawiera funkcje do użytku dla sterowników urządzeń i TDI. NDIS mini port

służy utworzeniu dla TDI interfejsu do konkretnych urządzeń sieciowych.

Transporty TDI to sterowniki protokołów pracujące w trybie jądra. Przejmują IRP od klientów TDI i

przetwarzają żądania, które te IRP reprezentują. Możliwe, że będzie to wymagać połączenia z inną maszyną, w tej sytuacji do danych zawartych w IRP mogą być dodane nowe nagłówki (TCP, UDP, IP...). Retransmisja i odpowiadanie na żądania to niektóre z ról tej warstwy.

Klienci TDI są to sterowniki w trybie jądra, które zwykle implementują API sieciowe. API

sieciowe (Networking API) umożliwiają aplikacjom porozumiewanie się przez sieć, niezależniąc sposób połączenia od zastosowanego protokołu.

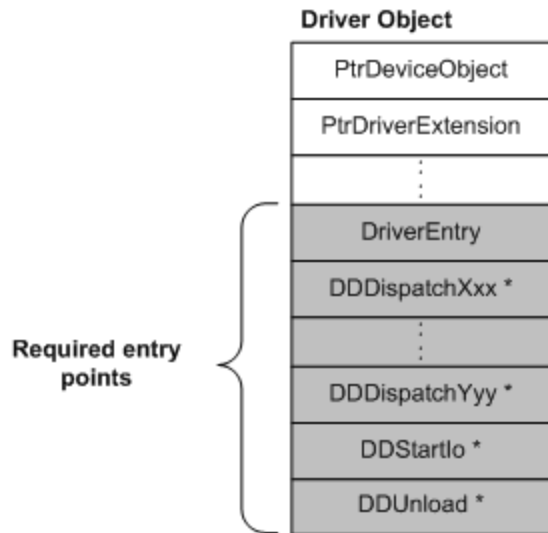
40. Na czym polega zagadnienie DLL-hell. Wymień sposoby pozwalające na uniknięcie tego problemu.

Jest to problem, który może się ujawnić przy używaniu dynamicznych bibliotek DLL. Najczęściej jest on spowodowany brakiem lub nieodpowiednią wersją biblioteki (przykładowo biblioteka DLL dostępna jest w trzech wersjach, nazywa się we wszystkich tak samo, ale funkcje przez nią udostępniane się różnią.

- **Global Assembly Cache** jest jednym z rozwiązań. Jest to część platformy CLR (czyli .NET). Przykładowo jeśli potrzebujemy dwóch bibliotek o tej samej nazwie (więc nie mogą obie być przechowywane w jednym miejscu na HDD), to można skorzystać z wirtualnego systemu plików należącego do tego rozwiązania. Z poziomu programu będzie to wyglądało tak jakbyśmy korzystali z dwóch różnych bibliotek, ale o tej samej nazwie.
- **Linkowanie statyczne** - biblioteka w konkretnej wersji jest wkompilowywana w program i aplikacja robi się bardziej stand-alone. Wadą jest brak możliwości aktualizacji/usunięcia błędów przez dostawcę biblioteki w przyszłości
- **Ochrona bibliotek** systemowych przed nadpisywaniem przez inne programy - *Windows File Protection*
- **Uruchamianie konfliktowych DLL-ów równocześnie** -dwie możliwości:
 - *Side-by-Side Component Sharing*- which loads separate copies of DLLs for each application that requires them (and thus allows applications that require conflicting DLLs to run simultaneously). This approach eliminates conflicts by allowing applications to load unique versions of a module into their address space, while preserving the primary benefit of sharing DLLs between applications (i.e. reducing memory use) by using memory mapping techniques to share common code between different processes that do still use the same module.
 - having different copies of the same DLLs for each application, both on-disk and in memory, placing the different versions of the problem DLL into the applications' folders, rather than a system-wide folder. However, [OLE](#) prevented this before Windows XP, because earlier versions of Windows had a single registry of COM objects for all applications.

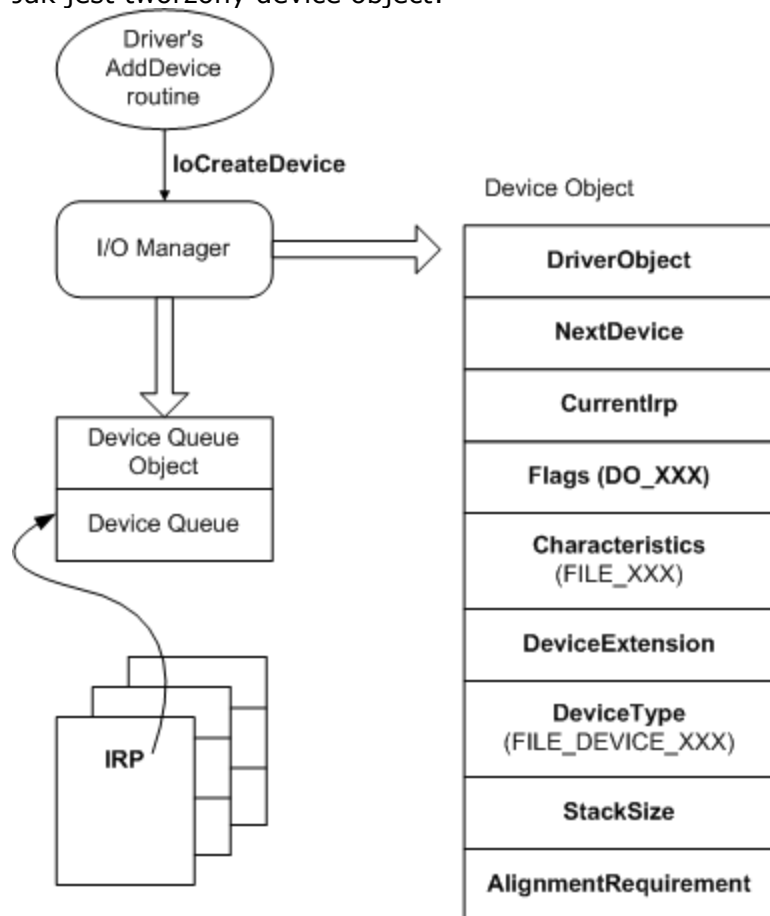
41. Wymień trzy podstawowe struktury systemowe wykorzystywane przy budowie i działaniu sterowników urządzeń i podaj ich znaczenie

- DRIVER OBJECT – reprezentuje obraz załadowanego sterownika.
- DEVICE OBJECT – reprezentuje urządzenie fizyczne lub logiczne w systemie i opisuje jego charakterystykę
- IRP (I/O Request Packet) – paczki z danymi, przesyłane między sterownikami.



Additional Standard Driver Routines	Lowest-level Drivers	Higher-level Drivers
Required	DDInterruptService	
	DDDpcForISR * or DDCustomDPC	
Sometimes required, depending on device and driver functionality	DDSynchCritSection	
	DDCancel *	
	DDAdapterControl * and/or DDControllerControl *	DDCancel *
Optional	DDIoTimer DDCustomTimerDPC DDReinitialize	DDIoCompletion* DDIoTimer DDCustomTimerDPC DDReinitialize

Jak jest tworzony device object:



42. Podaj różnice pomiędzy zadaniami (jobs), procesami, wątkami i włóknami.

Zadanie jest to nazwany, chroniony i współdzielony obiekt systemu, który pozwala na zarządzanie grupą procesów. Głównym celem zadania jest by grupa procesów mogła być zarządzana jako jedna całość.

Proces jest to wykonujący się program. Jeden proces może należeć do jednego zadania. Jest to kontener z zasobami używanymi podczas wykonywania programu.

Wątek jest to główna jednostka wykonawcza w systemie Windows. Pojedynczy wątek może należeć do jednego procesu, ale proces może zawierać wiele wątków (co najmniej jeden – wątek główny). Każdy wątek zawiera stan rejestrów procesora, dwa stosy (tryb kernela i chroniony osobno), prywatny obszar przechowywania TLS, unikatowy identyfikator oraz czasami dodatkowy kontekst bezpieczeństwa.

Włókno często bywa nazwane 'lekkim wątkiem'. Pozwala aplikacji na samodzielne szeregowanie zadań (przełączanie aktywnych włókien) zamiast zdawać się na globalny

dyspozytor. Włókno nie jest widzialne dla jądra, działa jedynie w trybie użytkownika. Może mieć prywatny obszar przechowywania FLS.

43. Jaka jest rola mechanizmu DPC i na czym on polega ?

- Odroczone wywołanie procedury (DPC) jest wykorzystane do wykonania kodu przy przejściu z wyższego poziomu IRQL na niższy
 - Sterownik (proces jądra) kolejkuje żądania,
 - wykonanie żądań z kolejki następuje w momencie zakończenia pracy wszystkich przerwań o wyższym IRQL
- Wykorzystane jest do generowania timeout'ów oraz przełączania wątków. Dzięki temu mechanizmowi operacje krytyczne (np. funkcje sterowników sprzętowych) są wykonywane szybciej niż operacje, które nie muszą być wykonywane od razu.

A **Deferred Procedure Call** (DPC) is a Windows [operating system](#) mechanism which allows high-priority tasks (e.g. an [interrupt handler](#)) to defer required but lower-priority tasks for later execution. This permits device drivers and other low-level event consumers to perform the high-priority part of their processing quickly, and schedule non-critical additional processing for execution at a lower priority.

DPCs are implemented by DPC objects which are created and initialized by the [kernel](#) when a [device driver](#) or some other kernel mode program issues requests for DPC. The DPC request is then added to the end of the system-wide DPC [queue](#). DPCs have three priority levels: low, medium and high. By default, all DPCs are set to medium priority. When the processor drops to an [IRQL](#) (interrupt request level) of Dispatch/DPC level, it checks the DPC queue for any pending DPCs and executes them until the queue is empty or some other interrupt with a higher IRQL occurs.

For example, when the clock interrupt is generated, the clock interrupt handler generally increments the counter of the current thread to calculate the total execution time of that thread, and decrements its [quantum](#) time remaining by 1. When the counter drops to zero, the thread scheduler has to be invoked to choose the next thread to be executed on that processor and dispatcher to perform a context switch. Since the clock interrupt occurs at a much higher IRQL, it will be desirable to perform this thread dispatching which is a less critical task at a later time when the processor's IRQL drops. So the clock interrupt handler requests a DPC object and adds it to the end of the DPC queue which will process the dispatching when the processor's IRQL drops to DPC/Dispatch level.

44. Wyjaśnić różnice w tworzeniu połączeń komunikacyjnych za pomocą gniazd z użyciem protokołu UDP i TCP.

TCP – protokół działający w trybie połączeniowym. Połączenie za jego pomocą odbywa się w trzech etapach:

- zestawienie połączenia (zestawienie ścieżek po których będą przepływać dane)
- przesyłanie danych
- zakończenie połączenia.

UDP – protokół działający w trybie bezpołączeniowym, za jego pomocą przesyłane są tzw. datagramy. Jest dość zawodny – pakiety mogą docierać do odbiorcy różnymi ścieżkami, w różnej kolejności. Lepszy od TCP, gdy chcemy przesłać pojedynczy datagram.

45. Wyjaśnij następujące pojęcia MBR, VBR, DDK, IDT, GDTR.

- **MBR** – Master Boot Record. Jest to pierwszy sektor na dysku zawierający specjalny program, który jest odczytywany przy starcie komputera do pamięci RAM i którego zadaniem jest przeszukanie tablicy partycji w celu przejścia do VBR.

The MBR contains 4 entries about the locations and type of the logical partitions (e.g NTFS, FAT) on that physical hard drive, one of which is "active" and small piece of code (446 bytes) called the primary bootloader. The bootloader is 446 bytes long and the information describing the partitions is 64 bytes long (total 510 bytes). The final two bytes of the first sector, sometimes known as the "magic number", is the hex value of 55AA.

MBR tells the computer the location and nature of the first active partition, which is commonly at **Sector 63**. A detailed description of the MBR, including its offsets is available here.

The first sector in a partition – which the MBR points to – is known as the **volume boot sector, boot block, volume boot record** or by some companies as the "BPB", and contains information about the partition, including:

- Block size
- Size of the partition (size in blocks)
- The volume serial number
- The type of partition (e.g NTFS/FAT, etc)
- Where the MFT is (if its an NTFS)
- Where the MFT Mirror is (if its an NTFS)
- Location of the **NTLDR** or NTLoader (discussed later) – normally be straight after at sector 64.

The last sector of the partition is a mirror of the volume boot sector, and can be used for data recovery purposes.

As Sector 1 to Sector 62 are not used, they can often be blank, however manufacturers like HP and Dell sometimes write information about the machine in there, e.g serial numbers, model, etc. These could, in theory, be relevant forensic artifacts.

- **VBR** – Volume Boot Record. Podobny do MBR, ale znajduje się już na konkretnej partycji (na początku) i zawiera program, który powinien rozpocząć ładowanie systemu operacyjnego.
- **DDK** – Driver Development Kit. Zestaw narzędzi pozwalających na tworzenie sterowników urządzeń (i nie tylko) na pewną platformę.
- **IDT** – Interrupt Descriptor Table. Jest to tablica deskryptorów znajdująca się w pamięci RAM, w której zapisane są selektory, które z kolei mają indeksować tablicę GDT, która wskazuje segment, w jakim znajduje się procedura obsługi przerwania.
- **GDTR** – Global Descriptor Table Register. Jest to 48-bitowy rejestr wskazujący położenie w pamięci tablicy GDT (32 bity) oraz limit rozmiaru tej tablicy (16 bitów).

46. Wymień 5 różnych systemów plików.

NTFS, FAT32, ReiserFS, BeFS, ext2/3/4, Reiser4, XFS, JFS, HFS+ (Apple), UFS (*BSD), ZFS (Solaris), ISO 9660, also referred to as CDFS (Compact Disc File System)

47. Wyjaśnij pojęcie spoofing'u i podaj sposoby obrony.

Spoofing jest to technika ataków sieciowych polegająca na przesyłaniu pakietów preparowanych, zmodyfikowanych lub uszkodzonych. Przykładem może być przechwycenie pakietu i zmiana jest odbiorcy lub danych, które są w nim zawarte.

Sposoby ochrony:

- brama internetowa może odrzucać pakiety spoza sieci, które mają adres źródłowy z tej sieci,
- może też odrzucać pakiety z sieci, które mają adres spoza sieci,
- TCP negocjuje ze zdalną maszyną sekwencję cyfr, dzięki czemu wie, że przesyłane dane są częścią nawiązanego połączenia.
- korzystanie z protokołów szyfrujących (SSL...)

Arp-spoofing - rozgłaszamy często swój komputer(mac swojej sieciowki) jako ip używane już przez kogoś. Router zapisuje w tablicy arp, że teraz tamten to my i to my dostajemy jego pakiety. Rozgłaszamy też, że router to my, i dostajemy od tamtego. włączamy forward pakietów, bo inaczej tamten straci połączenie. Teraz mamy ładne mitm i wszystko sniffujemy.

Sposób obrony - statyczne tablice arp; różne programy do śledzenia zmian tych tablic i zgłaszania podejrzanym zmian.

48. Omówić mechanizm przełączania stosu używany w trakcie przyjmowania przerwania.

Mechanizm ten jest identyczny jak w przypadku wołania funkcji za pomocą furtek. Potrzeba jego użycia zachodzi, gdy program wołany jest bardziej uprzywilejowany niż program wołający.

Przebiega następująco:

- nowe wartości rej. SS i ESP przechowywane są w TSS (Task State Segment)
- są one ładowane do SS i ESP, a stare są zachowywane na nowym stosie
- ze starego stosu na nowy kopiowane są parametry podprogramu
- na stosie zapisywany jest ślad (zawartość CS i EIP).

49. Przedstawić klasyfikację maszyn wirtualnych i opisać ich charakterystyki

Klasyfikacja VM



Maszyny wirtualne to m.in.:

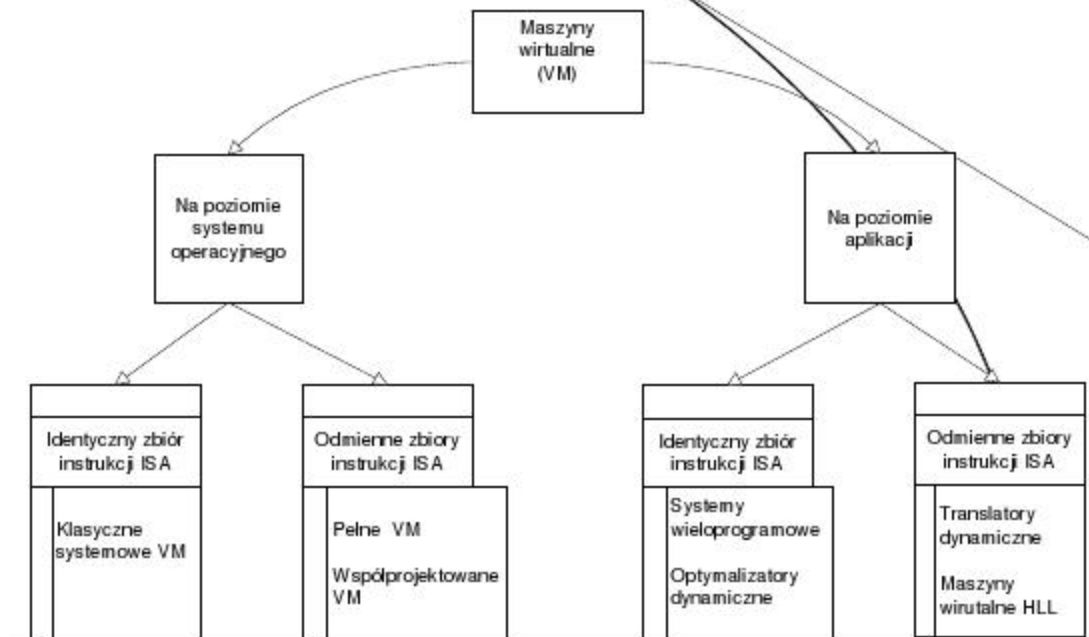
- * interpretery, szczególnie interpretery kodu bajtowego
- * kompilatory JIT
- * emulatory rzeczywiście istniejącego sprzętu, np. emulatory konsol

Maszyny wirtualne można podzielić na maszyny wirtualne procesowe i systemowe. VM na poziomie procesu dostarcza **ABI (Application Binary Interface – rozdziela procesy od systemu operacyjnego)** dla aplikacji, a systemowa dostarcza zbioru **ISA (Instruction Set Architecture – rozdziela sprzęt od pozostałej części systemu)** dla systemu operacyjnego i aplikacji.

Procesowe maszyny wirtualne działają jako normalny proces w systemie, są z nim tworzone i usuwane. Ich celem jest przede wszystkim uniezależnienie programowania od platformy, by programy były przenośne mimo różnic sprzętowych (lub też programowych) w maszynach. Przykładem procesowej maszyny wirtualnej może być Java Virtual Machine. Systemowe maszyny wirtualne zezwalają na współdzielenie zasobów sprzętowych między maszyny wirtualne, z których każda może być uruchomiona z innym systemem operacyjnym. Przykładem jest VirtualBox. Wadą tego rozwiązania jest to, że wydajność

systemu zainstalowanego na maszynie wirtualnej jest niższa.

Klasyfikacja VM



0. Wskaż zagadnienia, które należy: wprowadzić, zmodyfikować, usunąć z tematyki wykładu z przedmiotu Oprogramowanie Systemowe. Odpowiedz uzasadnij ☺

usunąć zagadnienia związane z siecią, sterownikami, systemami plików, rodzajami plików wykonywalnych, maszynami wirtualnymi, zarządzaniem pamięcią, obsługą przerwań, wywoływaniem procedur systemowych, ładowaniem plików wykonywalnych do pamięci...(o czymś zapomniałem?) bo za trudne ;P

wprowadzić więcej o nowych systemach takich jak windows vista - np. user-mode drivers :P

I więcej HTML-a i niech ten przedmiot będzie prowadzony przez KIW XD

50. Jaka jest rola mechanizmów APC i na czym ona polega?

An asynchronous procedure call (APC) is a function that executes asynchronously in the context of a particular thread. When an APC is queued to a thread, the system issues a software interrupt. The next time the thread is scheduled, it will run the APC function. An APC generated by the system is called a kernel-mode APC. An APC generated by an application is called a user-mode APC. A thread must be in an alertable state to run a user-mode APC.

Each thread has its own APC queue. An application queues an APC to a thread by calling the QueueUserAPC function. The calling thread specifies the address of an APC function in the call to QueueUserAPC. The queuing of an APC is a request for the thread to call the APC function.

When a user-mode APC is queued, the thread to which it is queued is not directed to call the APC function unless it is in an alertable state. A thread enters an alertable state when it calls the SleepEx, SignalObjectAndWait, MsgWaitForMultipleObjectsEx, WaitForMultipleObjectsEx, or WaitForSingleObjectEx function. If the wait is satisfied before the APC is queued, the thread is no longer in an alertable wait state so the APC function will not be executed. However, the APC is still queued, so the APC function will be executed when the thread calls another alertable wait function.

APC wykorzystywane jest przy asynchronicznej komunikacji między aplikacją użytkownika a sprzętem.

51. Czym są do czego służą tzw. zbiory robocze (working sets)?

Working set to pamięć, która nie podlega przeniesieniu na dysk ze względu na swoje krytyczne znaczenie dla działania programu.

The *working set* of a program is a collection of those pages in its virtual address space that have been recently referenced. It includes both shared and private data. The shared data includes pages that contain all instructions your application executes, including those in

your DLLs and the system DLLs. As the working set size increases, memory demand increases.

A process has an associated minimum working set size and maximum working set size. Each time you call **CreateProcess**, it reserves the minimum working set size for the process. The virtual memory manager attempts to keep enough memory for the minimum working set resident when the process is active, but keeps no more than the maximum size.